MODULE 1

OPERATING SYSTEMS

(Common to CS/CD/CM/CR/CA/AD/AI/CB/CN/CC/CU/CI/CG)

Course Code	**PCCST403**	**CIE Marks**	40
Teaching Hours/Week (L: T:P: R)	3:1:0:0	**ESE Marks**	60
Credits	4	**Exam Hours**	2 Hrs. 30 Min.
Prerequisites (if any)	None	**Course Type**	Theory

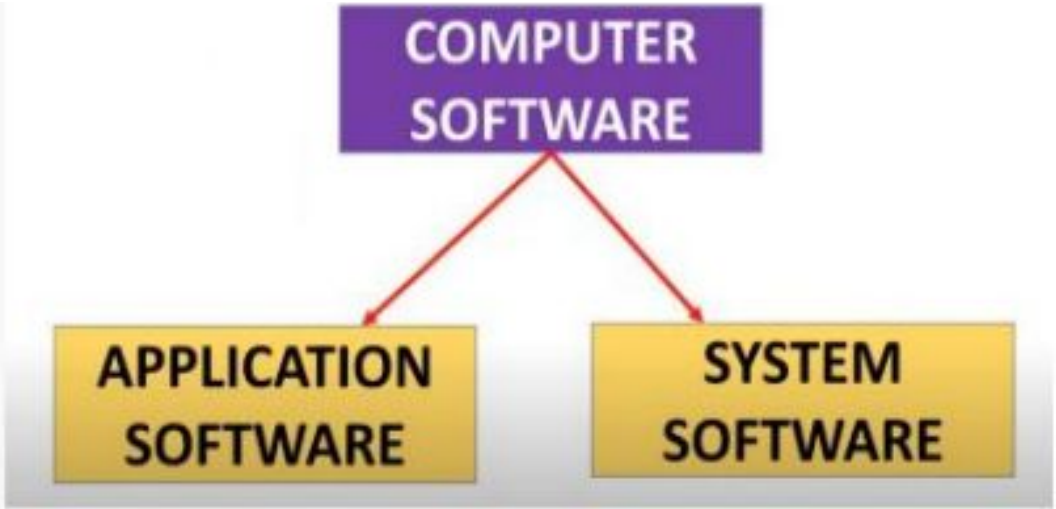
Dilsha K P

- **Introduction to Operating Systems**
- **Operating System Services**
- **Overview of Operating Systems and Kernels,**
- **Linux Versus Classic Unix Kernels**

Introduction to Operating system

An **operating system is a program that manages a computer's hardware.**

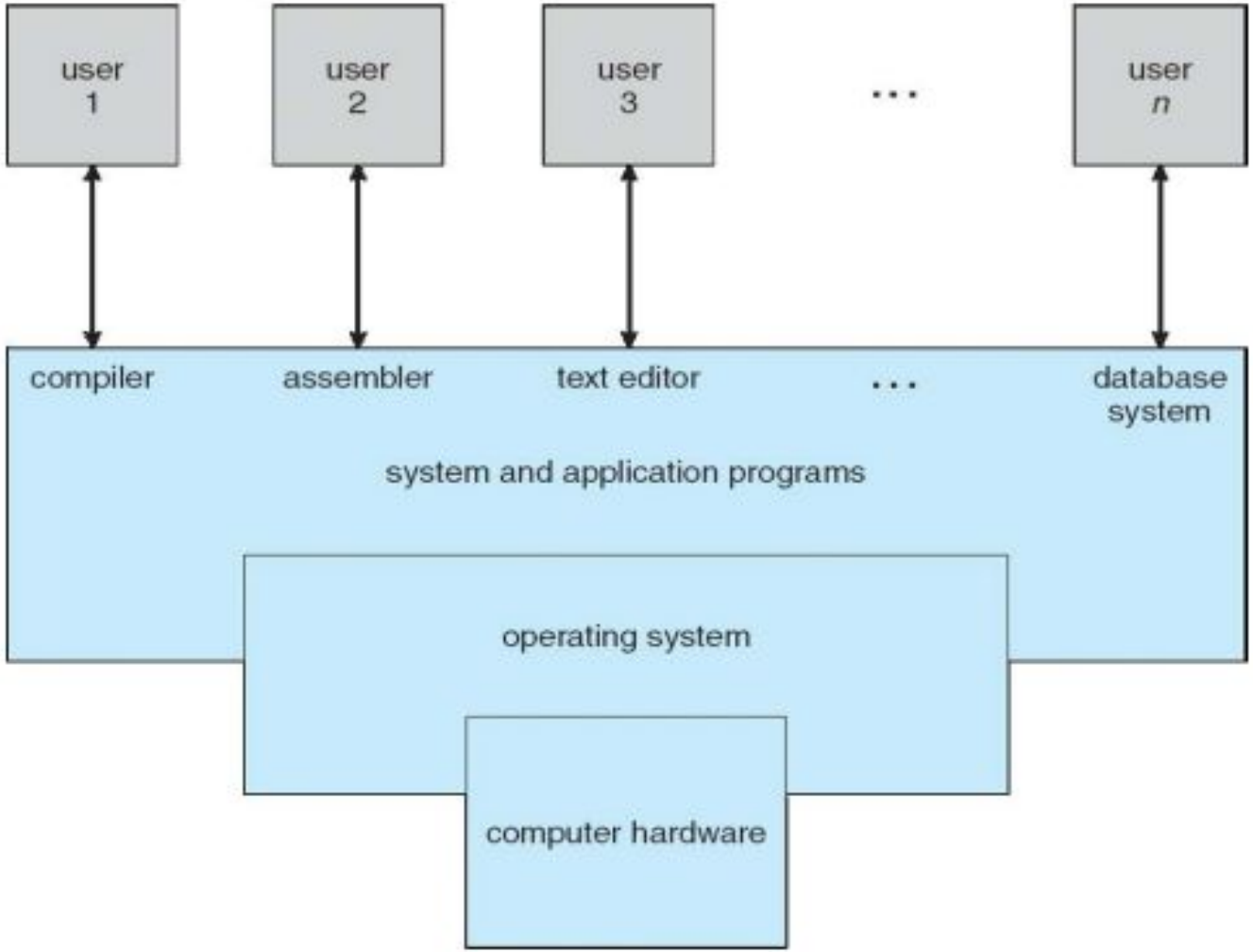
An Operating System is a program that acts as an **intermediary** between a **user** of a computer and the computer **hardware**.



Application Soft wares are programs that help the user to perform specific task.
Eg: Word processors, web browsers, database systems, video games

System Software is a set of programs that control and manage the operations of computer hardware. It also helps application programs to execute correctly.

Computer system can be divided into four components

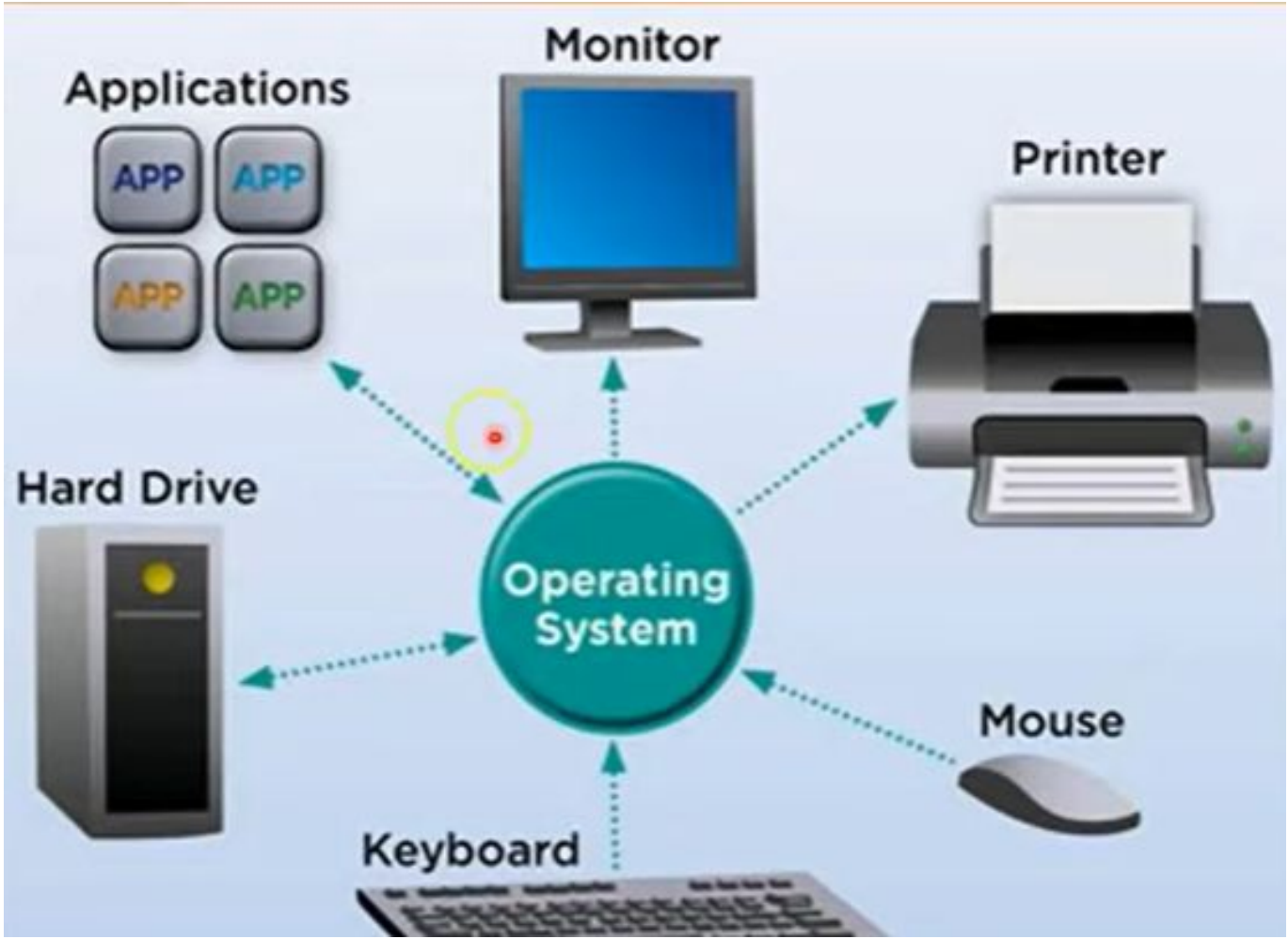


Computer system can be divided into four components

Computer System Components

1. **Hardware**
    - Provides the basic computing resources
    - Examples: CPU, memory, input/output devices
2. **Operating System**
    - Controls and manages the hardware
    - Coordinates hardware use among multiple users and applications

1. **Application Programs**
    - Use system resources to solve user tasks
    - Examples: Word processor, web browser, database software, games
2. **Users**
    - People, machines, or other computers that use the system


Monitor
Applications
APP
APP
APP
APP
Printer
Hard Drive
Operating System
Mouse
Keyboard

Operating System as a Resource Allocator

- Manages and controls all hardware and software resources
- Resolves conflicting requests from programs and users
- Ensures resources are used efficiently and fairly

Operating System as a Control Program

- Controls the execution of programs and ensures correct operation
- Prevents errors and avoids improper use of the system
- Manages and executes user and application programs
- Makes the computer system easy and convenient to use
- Helps users solve their tasks more efficiently
- Ensures hardware resources are used effectively

Operating System services

Prepared by Dilsha K P

OS provides essential services that allow applications to run effectively and ensure smooth interaction between hardware and users. Here are the main services offered by an OS

1. **Program Execution**
2. **Input/Output Operations**
3. **File System Manipulation**
4. **Communication**
5. **Error Detection**
6. **Resource Allocation**
7. **Protection**
8. **Accounting**

Program Execution

- ★ Operating System manages how a program is going to be executed.
- ★ It Loads a program into memory.
- ★ Executes the program.
- ★ Handles program's execution.
- ★ Provides a mechanism for process synchronization.

Input/Output Operations

- Manages input/output devices and operations.
- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required

File System Manipulation

- A file represents a collection of related information.
- Computers can store files on the disk (secondary storage), for long-term storage purpose
- A file system is normally organized into directories for easy navigation and usage.
- The OS manages file operations by granting permissions like read-only, read-write, or denied.
- It provides interfaces for creating/deleting files and directories.
- The operating system also supports file backups.

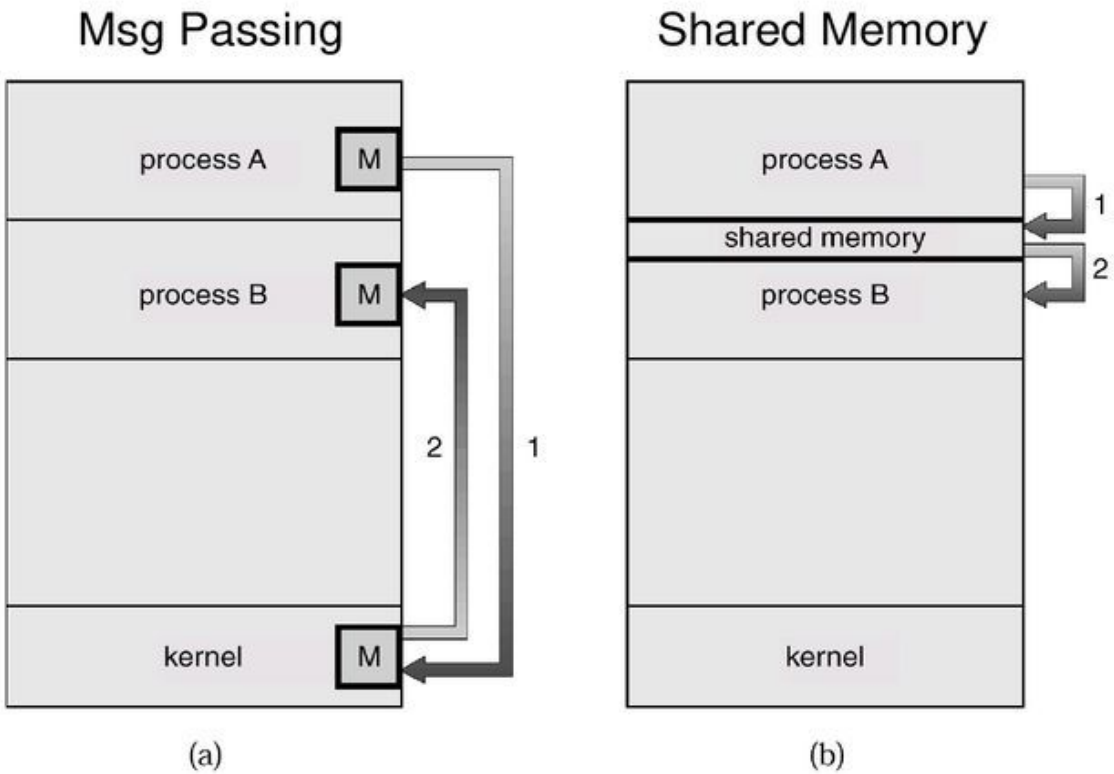
Communication

- The Operating system manages the communication between processes
- Communication between processes includes data transfer among them.
- The processes may reside on the same computer or different computers.
- If on different computers, communication is enabled through a network connection.

Methods of Communication:

- Shared Memory: Processes share a common memory space for communication.
- Message Passing: Processes exchange information via messages

Communication Models



Error Detection

- The Operating System also handles the error occurring in the CPU,Input-Output devices,Memory etc.
- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.
- It also ensures that an error does not occur frequently and fixes the errors.

Resource Allocation

- The OS manages and coordinates the sharing of system resources (CPU, memory, I/O devices) among various processes running on the system.
- Ensures that processes access resources without interfering with each other, providing a stable and multitasking environment.
- The OS allocates CPU time to different processes using CPU scheduling algorithms.
- The OS also ensures the proper use of all the resources available by deciding which resource to be used by whom

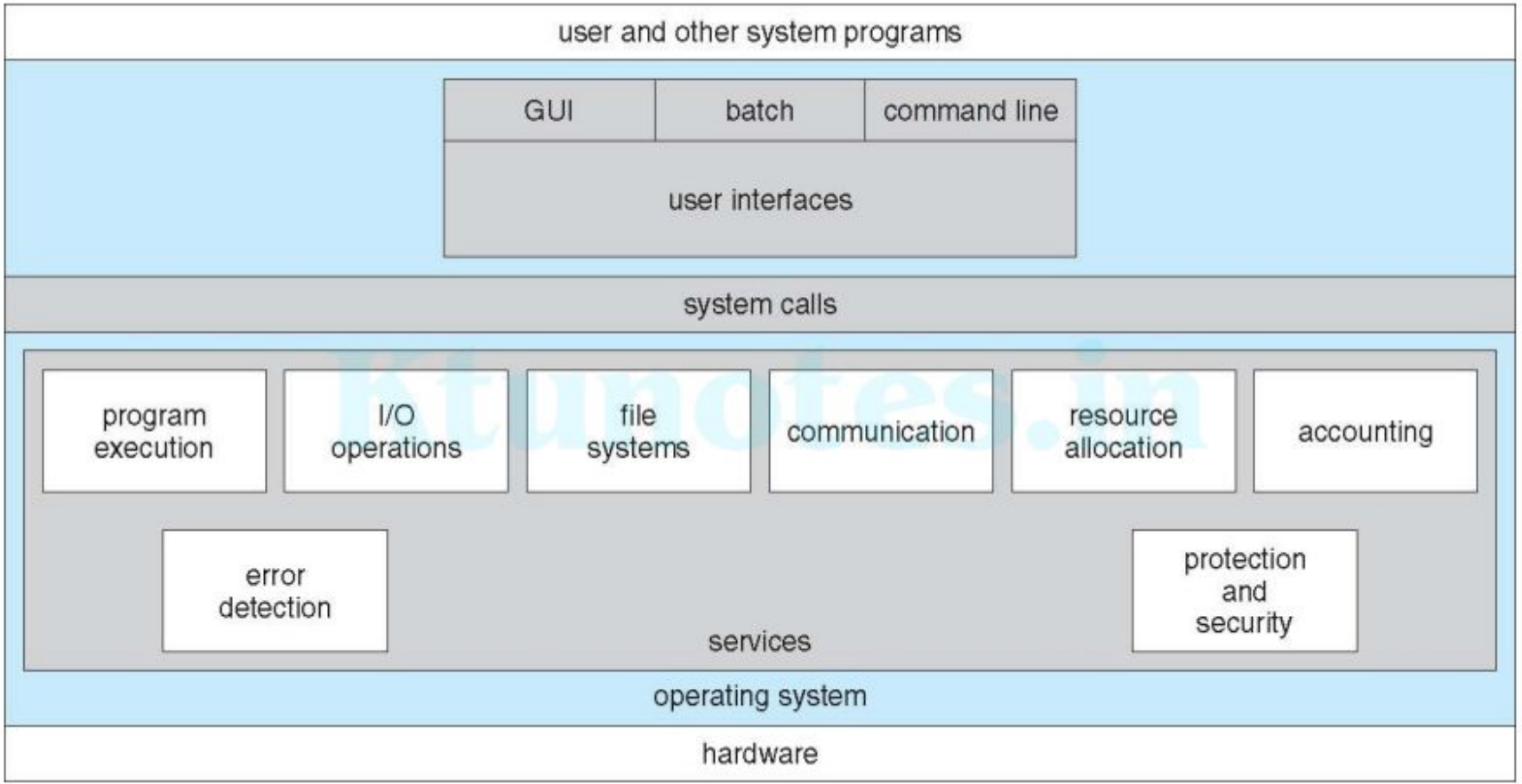
Protection

- The OS regulates and controls all access to system resources, ensuring efficient and secure usage.
- The OS prevents unauthorized access to external I/O devices
- The OS provides authentication mechanisms (e.g., passwords) to ensure only authorized users can access resources.

Accounting

- The OS keeps a record of how much CPU time, memory, disk space, and other resources are used by each process or user.
- The OS records how much of each resource is used and by whom
- helping to monitor system performance

A view of OS services



Overview of Operating Systems and Kernels

Prepared by Dilsha K P

What is an Operating System

- An Operating System (OS) is the software that manages computer hardware, and software, and provides services for computer programs.
- It acts as an intermediary between the user and the hardware, making it easier for users to interact with the system without needing to understand the complex details of the hardware.

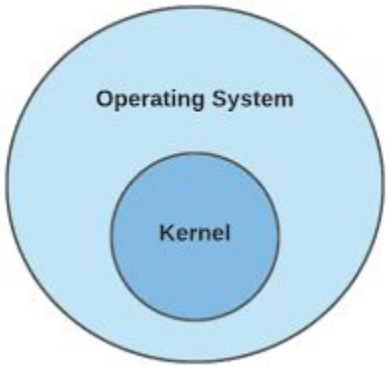
operating system includes only the essential software needed for system control and management. This includes:

1. The **kernel**
2. **Device drivers**
3. **Boot loader**
4. **Command shell or user interface**
5. **Basic system and file utilities**

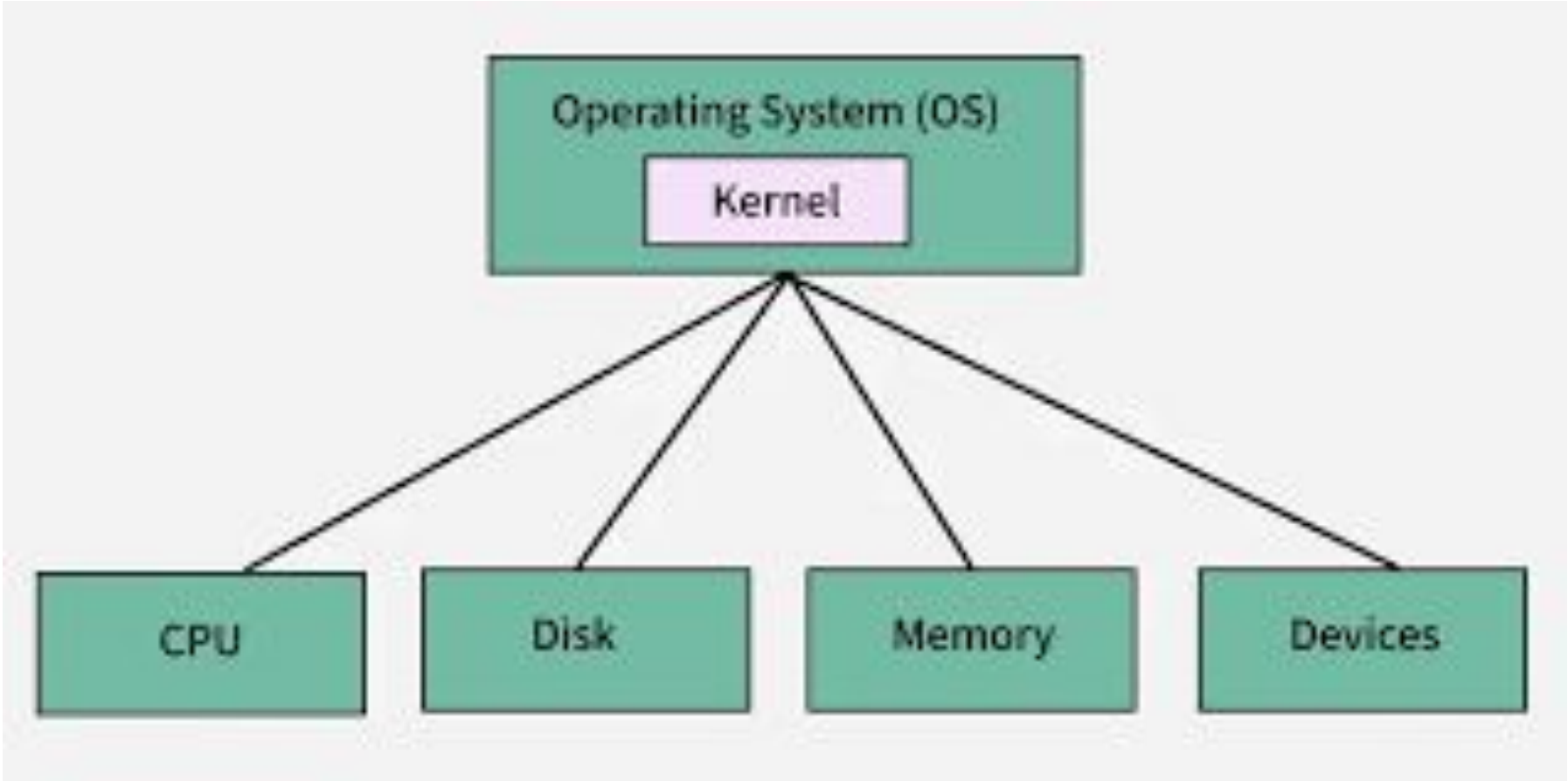
1. Kernel

1. The **kernel** is the core part of an operating system.
2. It manages communication between the hardware and the software.
3. Acts as a bridge between software applications and hardware.

4. It manages:
   Hardware (CPU, memory, I/O devices)
   System resources
   Process scheduling, multitasking



5. It provides a user interface, file system management, network services, and various utility applications that allow users to interact with the system.


Operating System (OS)
Kernel
CPU
Disk
Memory
Devices

2. Device Drivers

- **Device drivers** are specialized programs that allow the OS to communicate with hardware devices.

- Device Drivers depend upon the Operating System's instruction to access the device and perform any particular action.
- After the action, they also show their reactions by delivering output or status/message from the hardware device to the Operating system.
- For example, a printer driver tells the printer in which format to print after getting instruction from OS,

3. Boot Loader

- A **boot loader** is the small program that loads the operating system when the computer starts.
- Steps:
    1. Power on
    2. BIOS/UEFI runs
    3. Boot loader is loaded from storage
    4. Boot loader loads kernel into memory

4. Command Shell or User Interface

- This is how users interact with the system.
- Could be:
    - **Command-line interface (CLI)** → e.g., Bash, PowerShell
      Graphical user interface (GUI) → e.g., Windows desktop
- Allows users to run programs, manage files, and control the system.

5. Basic System and File Utilities

- These are built-in tools for managing the system and files.
- Examples:
  - **File utilities**: copy, move, delete, rename, search
  - **System utilities**: task manager, disk management, system monitor
- Essential for everyday operations and system maintenance.

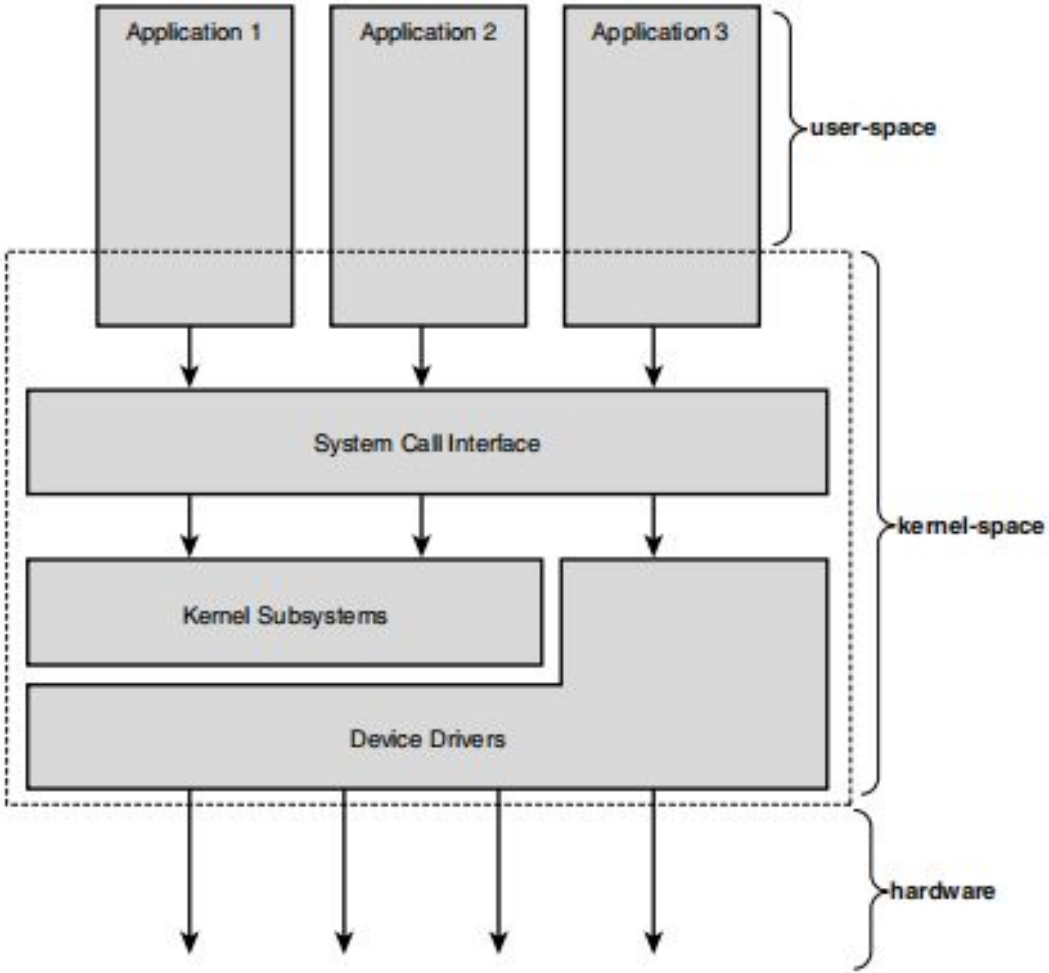


Figure 1.1 Relationship between applications, the kernel, and hardware.

Kernel

- The kernel is the most important and deepest part of the operating system.
- While the user interface is the outer layer, the kernel is the core.
- It provides critical services, manages hardware, and distributes system resources.
- The kernel is sometimes called the **core**, **supervisor**, or **internals** of the OS.

- The **kernel** is the **core component** of an operating system
- manages system resources and provides essential services to other parts of the operating system and user applications.
- It provides an **interface between application and hardware.**
- It converts the user query into the machine language.
- The main purpose of a kernel is to manage memory, disk and task.
- Without a kernel, an operating system cannot be run.
- Monolithic and Microkernels are the types of kernel.

A typical kernel includes:

- **Interrupt handlers** (for hardware signals)
- **Scheduler** (shares CPU time among programs)
- **Memory management system** (handles process memory)
- **System services** (networking, IPC, etc.)

Interrupt handlers: Small OS routines (**OS routines** means **small functions or procedures inside the operating system**) that quickly respond to hardware or software interrupt signals.

Scheduler: OS component that decides which process gets CPU time and when.

Memory management system: Manages allocation, deallocation, and protection of memory for processes.

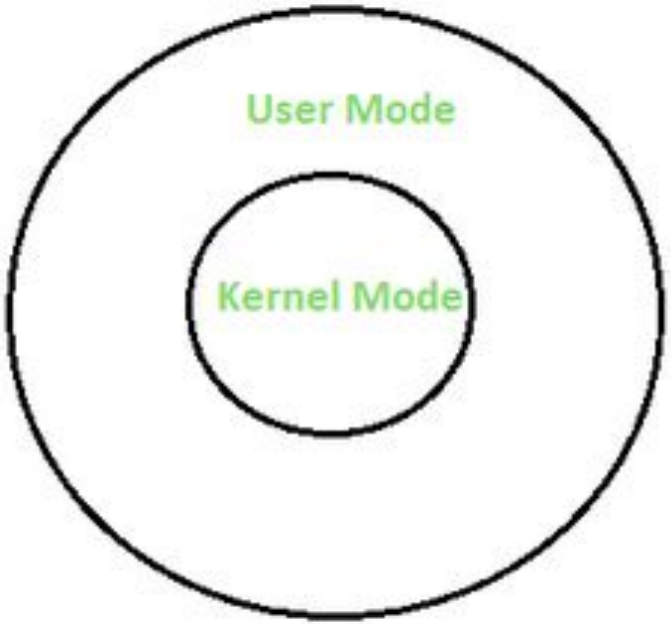
System services: Background OS services that support functions like networking, IPC, file handling, and device management.

- Modern systems run the kernel in a protected area called **kernel-space**, where it has full hardware access.
- Applications run in **user-space**, where access is limited to prevent system damage.
- When executing kernel instructions → **kernel mode**

- When running an application → **user mode**

The two modes of operation in an operating system are:

1. **User Mode**
2. **Kernel Mode / Privileged Mode**



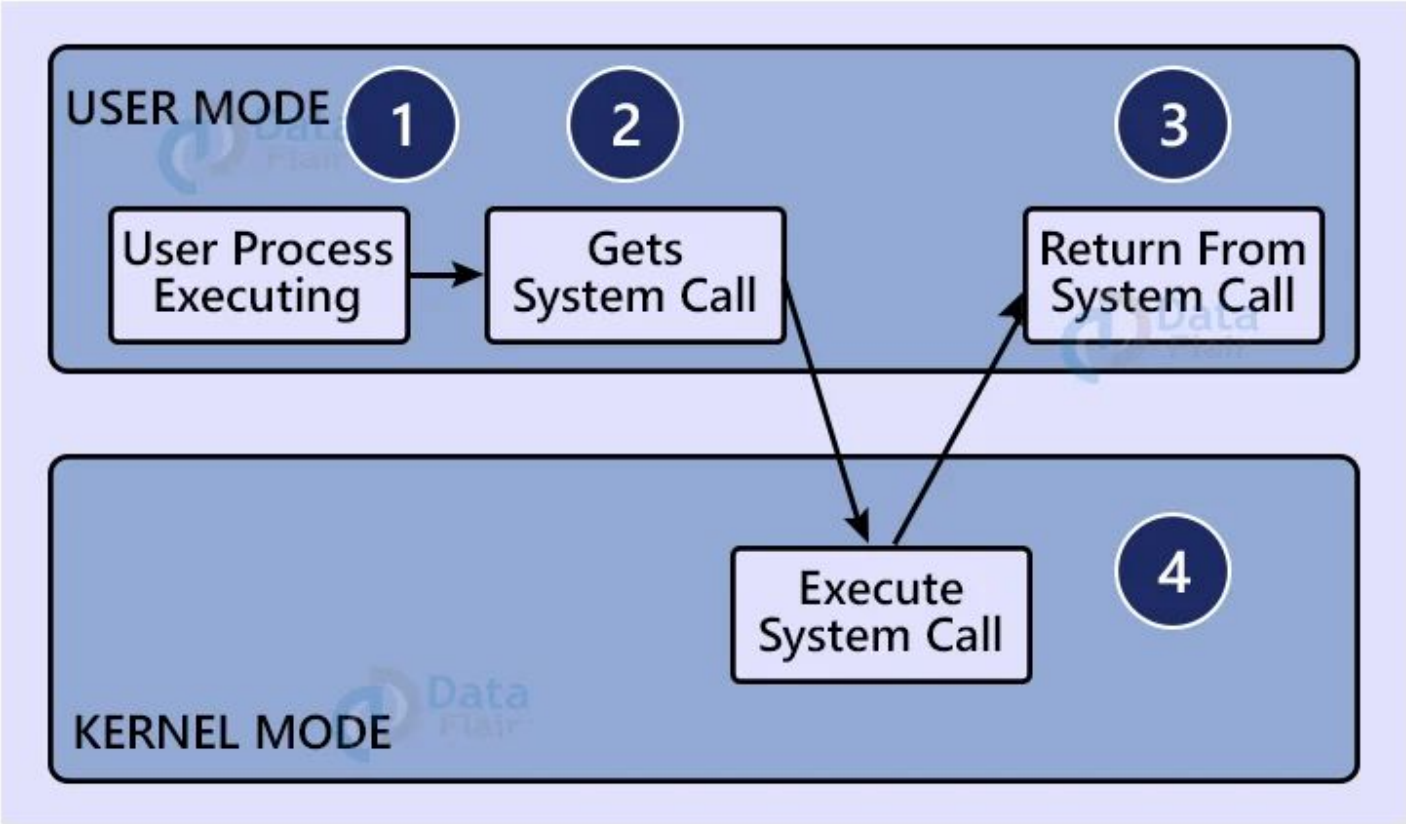
Kernel Mode	User Mode
In kernel mode, the program has direct and unrestricted access to system resources.	In user mode, the application program do not have direct access to system resources. In order to access the resources, a system call must be made.
OS has full control over the system and hardware, a crash in the kernel can lead to a system-wide failure, potentially causing the entire system to crash or become unstable.	if a program crashes, it typically only affects that specific user process, and the rest of the system remains unaffected.
Kernel mode is also known as the master mode, privileged mode, or system mode.	User mode is also known as the unprivileged mode, restricted mode, or slave mode.

System calls

Prepared by Dilsha K P

- The interface between a **process and an OS** is provided by system calls.
- A **system call** is a way for programs (or processes) to request services or resources from the OS
- These services include operations like file management, process control, memory allocation, and device handling.
- System calls are generally available as assembly language instructions

WORKING OF A SYSTEM CALL



Need for System Calls

Following are the reasons we need system calls:

- To read and write from files.
- To create or delete files.
- To create and manage new processes.
- To access hardware devices.

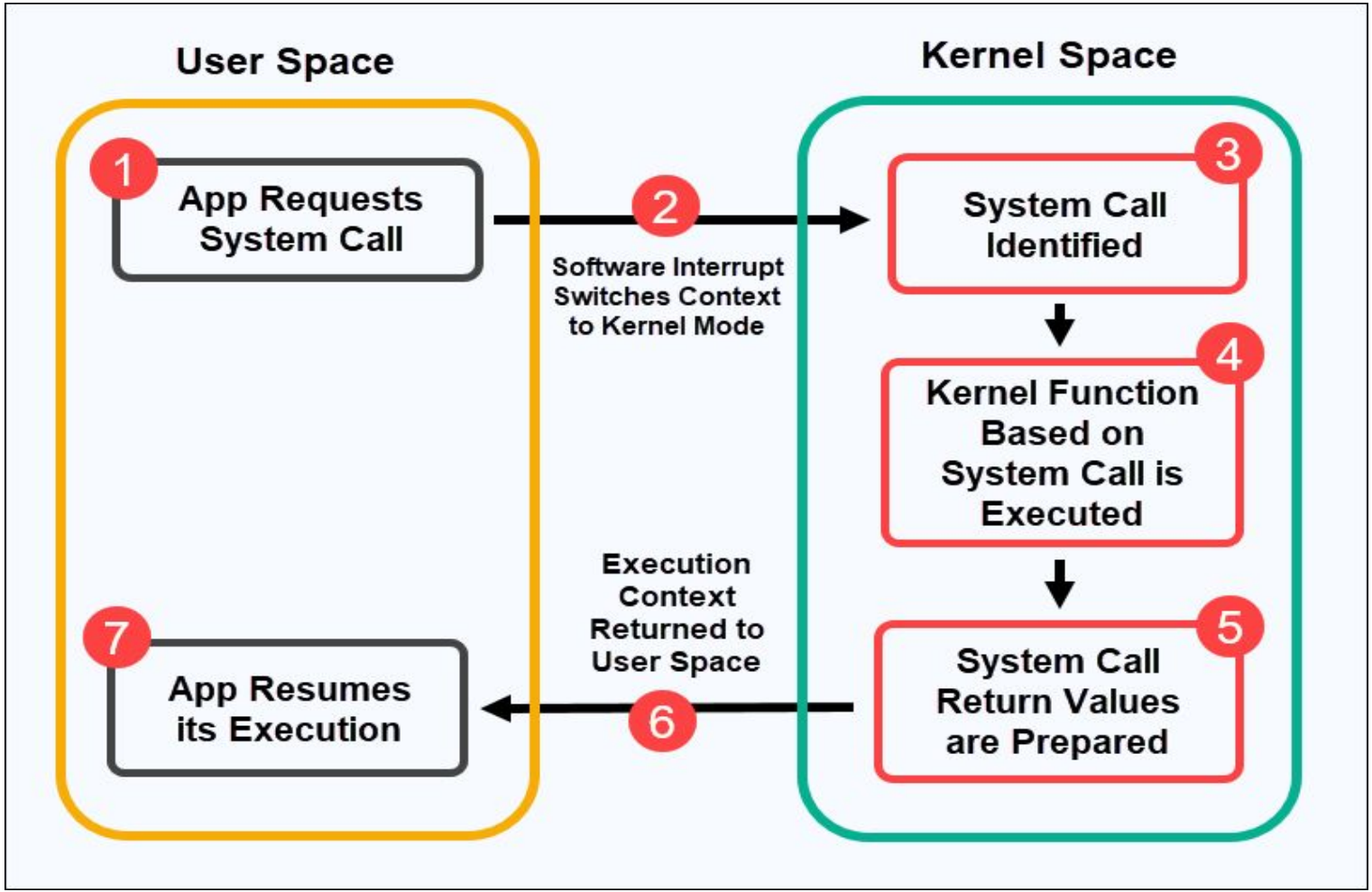
How a System Call Works

User Mode: A program running in user mode makes a system call (e.g., reading a file, allocating memory).

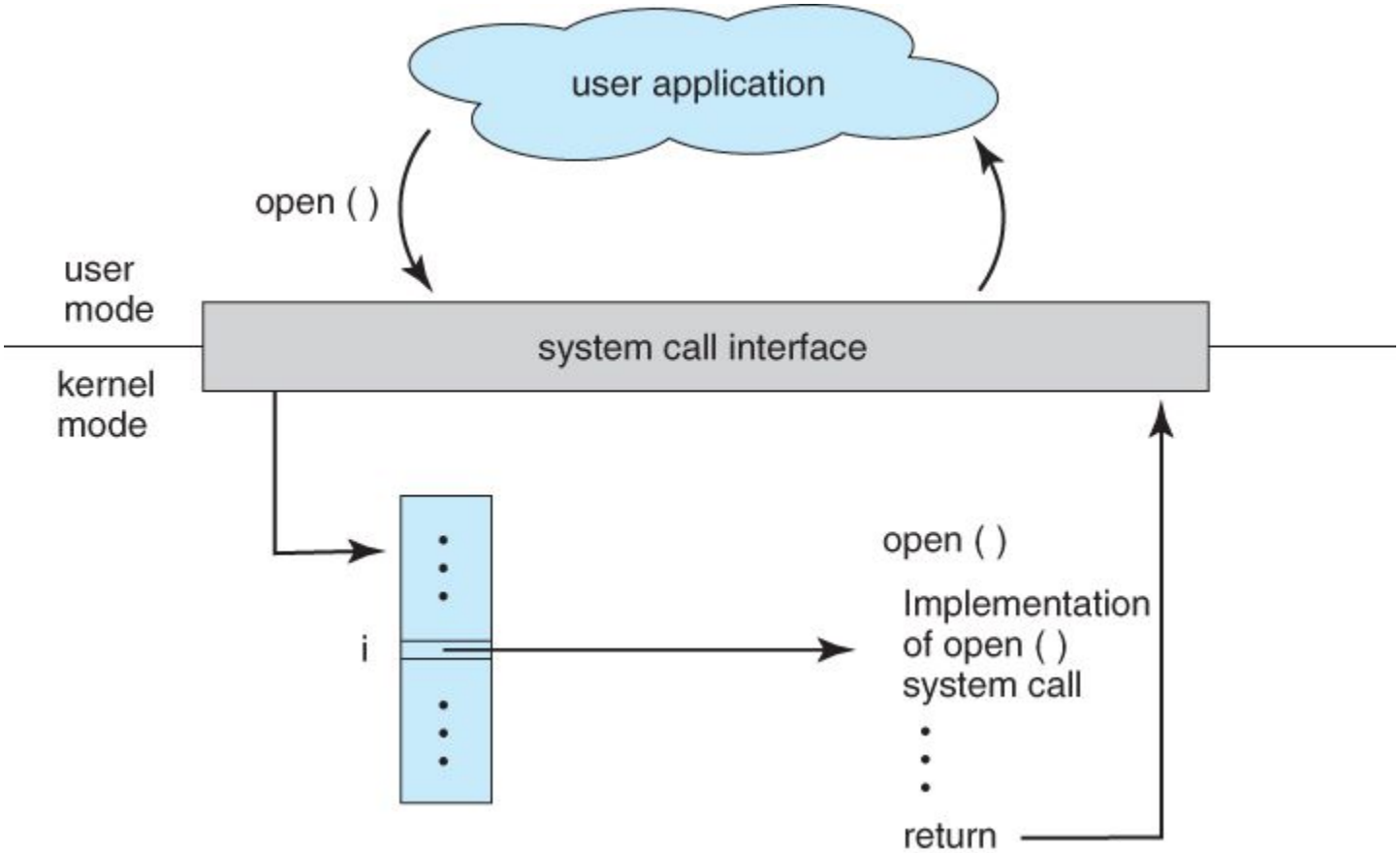
Context Switch to Kernel Mode: The operating system switches the CPU from user mode to kernel mode to handle the request. This switch is a context switch, as the OS saves the state of the user program and loads the kernel's state.

Kernel Executes the Request: The OS performs the requested task (e.g., accessing a device, managing files).

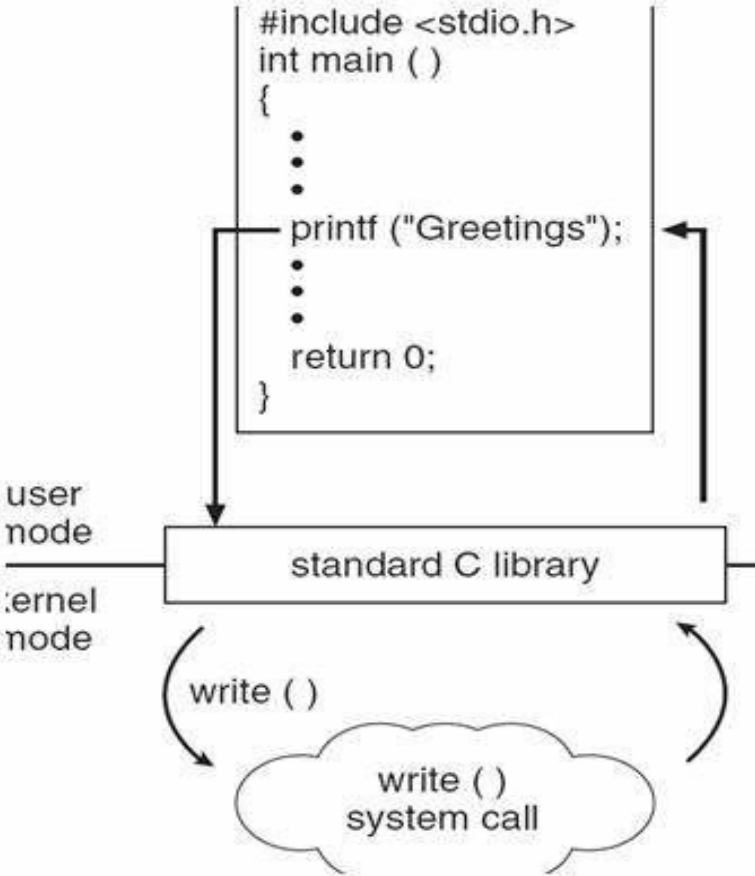
Return to User Mode: After the task is completed, the OS switches back to user mode, restoring the user program's state, and the program continues execution.


User Space
Kernel Space
1
App Requests System Call
2
Software Interrupt Switches Context to Kernel Mode
3
System Call Identified
4
Kernel Function Based on System Call is Executed
5
System Call Return Values are Prepared
6
Execution Context Returned to User Space
7
App Resumes its Execution

The handling of a user application invoking the open( ) system call



- Calls a function like printf().
- invokes the corresponding system call, such as write().
- **System Call (write())**: The kernel handles the system call and writes data to the output
- Receives the result from the kernel and passes it back to the user program.
- Continues execution



System Calls

Applications cannot directly access hardware.

They interact with the kernel using **system calls**.

Examples:

- printf() → formats output and eventually calls write() system call
- open() → directly calls the kernel.

When an application calls a system call, the kernel executes work **on behalf of the application** in **process context**.

Interrupts

- The kernel also responds to hardware signals called **interrupts**.
- When hardware needs attention, it interrupts the processor.
- Each interrupt has a number that helps the kernel run the correct interrupt handler.

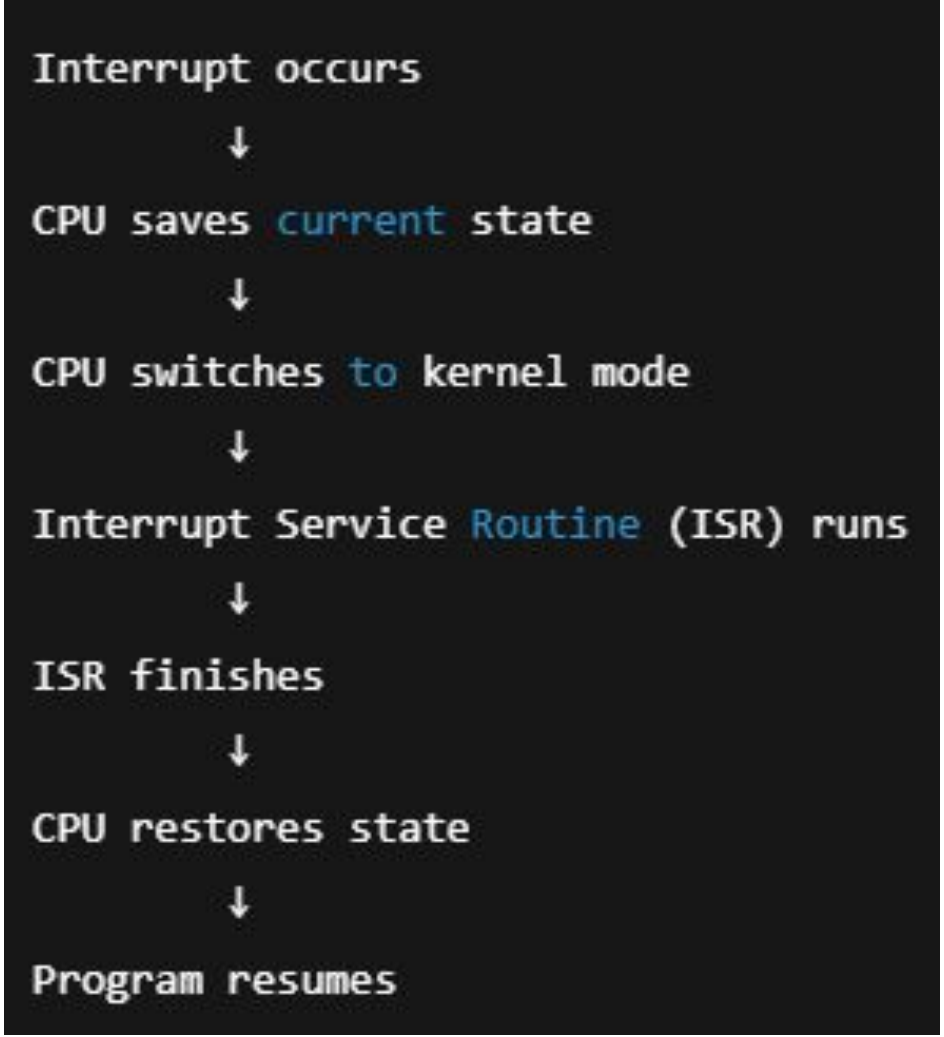
Example:

When a user types, the keyboard generates interrupts.

The kernel reads the key data and prepares for the next signal.

.

- An **interrupt** is a signal sent to the CPU that **stops the current execution** and forces the CPU to handle an important event immediately.
- Interrupts allow the OS to respond quickly to hardware events (keyboard input, timer tick, disk I/O completion).
- **Interrupt Service Routine (ISR)**is a small, fast piece of code that **handles an interrupt**—a signal sent by hardware or software to get the CPU's immediate attention.


Interrupt occurs
↓
CPU saves current state
↓
CPU switches to kernel mode
↓
Interrupt Service Routine (ISR) runs
↓
ISR finishes
↓
CPU restores state
↓
Program resumes

Example (Keyboard)

1. You press a key.
2. Keyboard controller sends interrupt.
3. Kernel reads interrupt number.
4. Kernel runs the keyboard interrupt handler.
5. Handler reads the key and tells keyboard it is ready for more.

Linux Versus Classic Unix Kernels

Prepared by Dilsha K

Linux Vs Unix

- ★ Linux and Unix are powerful multi-user operating systems, but differ in origin, licensing, architecture, community support, usage scope, file systems, shells, hardware compatibility, and security vulnerabilities.
- ★ Linux and Unix share a common ancestry.
- ★ Both use the same API (Application Programming Interface).
- ★ Due to this, they share several kernel design similarities.

Linux Kernel

- The Linux kernel is the core part of the Linux operating system.
- It was created by Linus Torvalds in 1991 and is open-source, meaning anyone can view, modify, and improve its code.

Classic Unix Kernels

- Classic Unix kernels refer to the original Unix systems developed in the 1970s
- These kernels formed the base of traditional commercial Unix systems.

Key Features of Classic Unix Kernels

Closed-source

- Classic Unix systems were developed by companies (e.g., AT&T, Sun Microsystems).
- Users could not freely view, modify, or distribute the source code.

Monolithic kernel structure

- The entire kernel runs as a single large program in one address space.
- Adding or changing features usually required recompiling the entire kernel.

Less modular than Linux

- Classic Unix kernels did not easily support dynamic loading of new functionality.
- Adding device drivers or new subsystems was harder and often static.

Limited hardware support

- Unix systems were often designed for specific hardware platforms.

Slower release cycles

- Updates and new features were released less frequently.
- Systems were stable but less flexible in adopting new technology quickly.

Very stable, commonly used in enterprise servers

- Widely deployed in critical server environments.

Key Features of the Linux Kernel

Open-source (GPL license)

- Anyone can access, modify, and redistribute the code under the GPL license.
- Encourages collaboration and rapid development.

Modular design: supports loadable kernel modules

- Drivers or features can be added or removed without restarting the system.
- Provides flexibility and easier maintenance.

Highly portable across many architectures

- Linux runs on PCs, servers, smartphones, and embedded devices.
- Supports x86, ARM, RISC-V, and many more.

Preemptive kernel for efficient multitasking

- The kernel can interrupt a running process to schedule higher-priority tasks.
- Improves system responsiveness and supports real-time applications.

Strong Symmetric Multiprocessing (SMP) support

- Efficiently uses multiple CPUs or cores for parallel processing.
- Improves performance for servers and multi-core systems.

Widely used across servers, desktops, Android devices, and embedded systems

- Linux’s flexibility and open development model make it suitable for a wide range of applications.

	Unix	Linux
Origin	Proprietary operating system	Open-source operating system
License	Commercial	Free and open-source
Kernel	Monolithic	Monolithic or hybrid
Software	Specialised and mature ecosystem	Vast collection of open-source software
Customisation	Limited	Highly customisable and adaptable
Cost	Expensive licensing costs	Generally free, with optional commercial support
Stability	Highly stable and reliable	Stable with active development and optimisation
Hardware	Supports various architectures	Supports various architectures
Community	Established support and resources	Active community-driven development

Monolithic Kernel vs Microkernel

Prepared by Dilsha K P

Kernel Design Approaches

There are two main kernel designs:

1. Monolithic Kernel
2. Microkernel

Monolithic Kernel

- Oldest and simplest kernel design.
- Implemented as **one large process** in a **single address space**.
- All services (drivers, file system, memory, scheduling) run in **kernel mode**.
- Communication is fast: **direct function calls**.

A **monolithic kernel** is a type of operating system architecture where **all essential services run in kernel mode**.

This includes:

- Device drivers
- File system management
- Memory management
- CPU scheduling
- System calls
- Networking

Everything runs inside **one large kernel space**.

The monolithic kernel manages the system resources between application and hardware of the system. But unlike microkernel, the user services and kernel services are implemented under same address space. This increases the size of the kernel further increases the size of operating system.

The monolithic kernel provides CPU scheduling, memory management, file management and other operating system functions through system calls. As user services and kernel services both reside in same address space, this results in the fast executing operating system.

One of the drawbacks of the monolithic kernel is if any one service fails entire system is crashed. If a new service is to be added in monolithic kernel, the entire operating system is to be modified.

Microkernel Design

- Kernel functionality split into separate processes called **servers.**
- Only essential components run in **kernel mode**.
- Other services run in **user space**.
- Communication happens through **message passing (IPC)**.

A **microkernel** is designed so that only the most essential parts run in kernel mode, such as:

- Memory management
- CPU scheduling
- Inter-process communication (IPC)

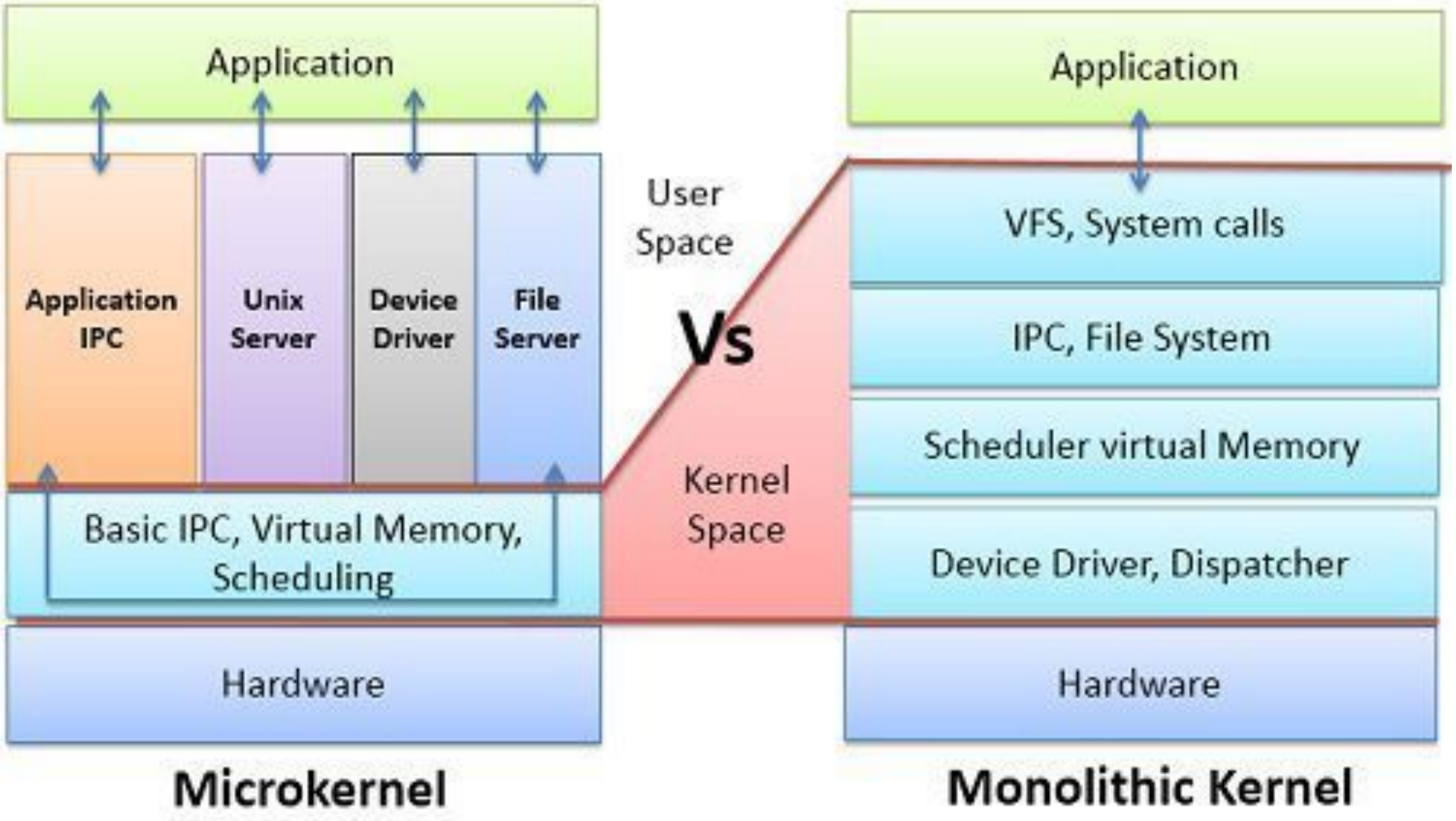
Other services like **device drivers, file systems, and networking** run in **user mode** as separate processes.

- Microkernel being a kernel manages all system resources.
- But in a microkernel, the user services and the kernel services are implemented in different address space.
- The user services are kept in user address space, and kernel services are kept under kernel address space.
- This reduces the size of the kernel and further reduces the size of operating system.

- Communication between application and hardware of the system, the microkernel provides minimal services of process and memory management.
- communication between the client program/application and services running in user address space is established through message passing.
- They never interact directly.
- This reduces the speed of execution of microkernel

- In a microkernel, the user services are isolated from kernel services so if any user service fails it does not affect the kernel service and hence Operating system remain unaffected. This is one of the advantages in the microkernel.
- The microkernel is easily extendable. If the new services are to be added, they are added to user address space and hence, the kernel space do not require any modification.
- The microkernel is also easily portable, secure and reliable.

Monolithic kernel	Microkernel.
In a monolithic kernel, user services and kernel services are kept in the same address space.	In a microkernel, user services and kernel services are kept in separate address space.
The monolithic kernel is larger in size.	The microkernel is smaller in size.
Execution speed is faster in the case of a monolithic kernel.	Execution speed is slower in the case of a microkernel.
The monolithic kernel is hard to extend.	The microkernel is easily extendable.
The whole system will crash if one component fails.	If one component fails, it does not affect the working of the microkernel.


Application
Application IPC
Unix Server
Device Driver
File Server
Basic IPC, Virtual Memory, Scheduling
Hardware
Microkernel
User Space
Vs
Kernel Space
Application
VFS, System calls
IPC, File System
Scheduler virtual Memory
Device Driver, Dispatcher
Hardware
Monolithic Kernel

Process concepts

Prepared by Dilsha K P

- A process is a program in execution
- It is a basic unit of work that can be scheduled and executed by the OS
- The OS helps you to create, schedule, and terminate the processes which are used by the CPU.
- A process created by the main process is called a child process.
- Process operations can be easily controlled with the help of PCB(Process Control Block).
- PCB is the brain of the process, which contains all the crucial information related to processing like process id, priority, state, CPU registers, etc.

Types of Processes

- **User Executed Process:** A process initiated and executed by a user.

  Examples :Text editors, browsers

- **System Executed Process :** A process initiated and executed by the OS.

  Examples:Memory management, device drivers

Process Creation

Prepared by Dilsha K P

Process Creation: How a Program Becomes a Process

When we run a program, the operating system must do several steps to transform the program stored on disk into a running process in memory.

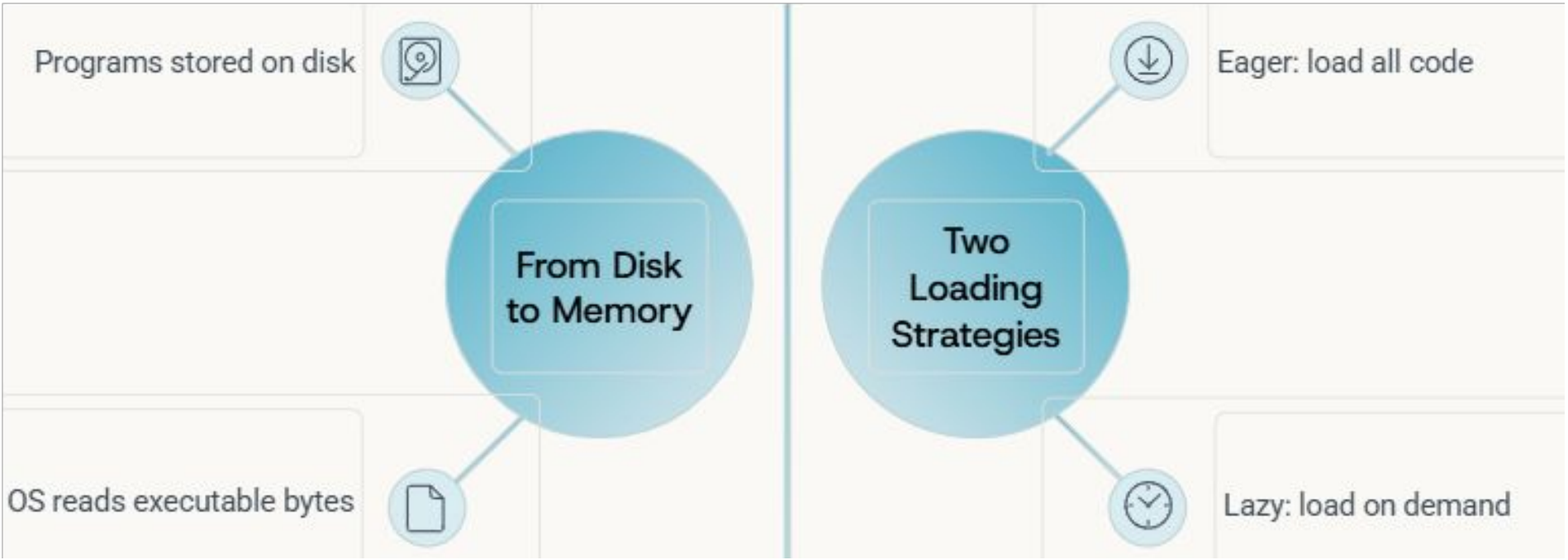
The steps are:

1. Loading the Program into Memory

- The program's **code and static data** are stored on disk in an executable file format.
- The OS reads this executable file and loads it into the **process address space**

  (A **process address space** is the range of memory addresses that a process is allowed to use while it is running.**).**

- Older OSes loaded the entire program at once **(eager loading)**.
- Modern OSes use **lazy loading** — only loading parts of the program when they are needed during execution.


Programs stored on disk
From Disk to Memory
OS reads executable bytes
Two Loading Strategies
Eager: load all code
Lazy: load on demand

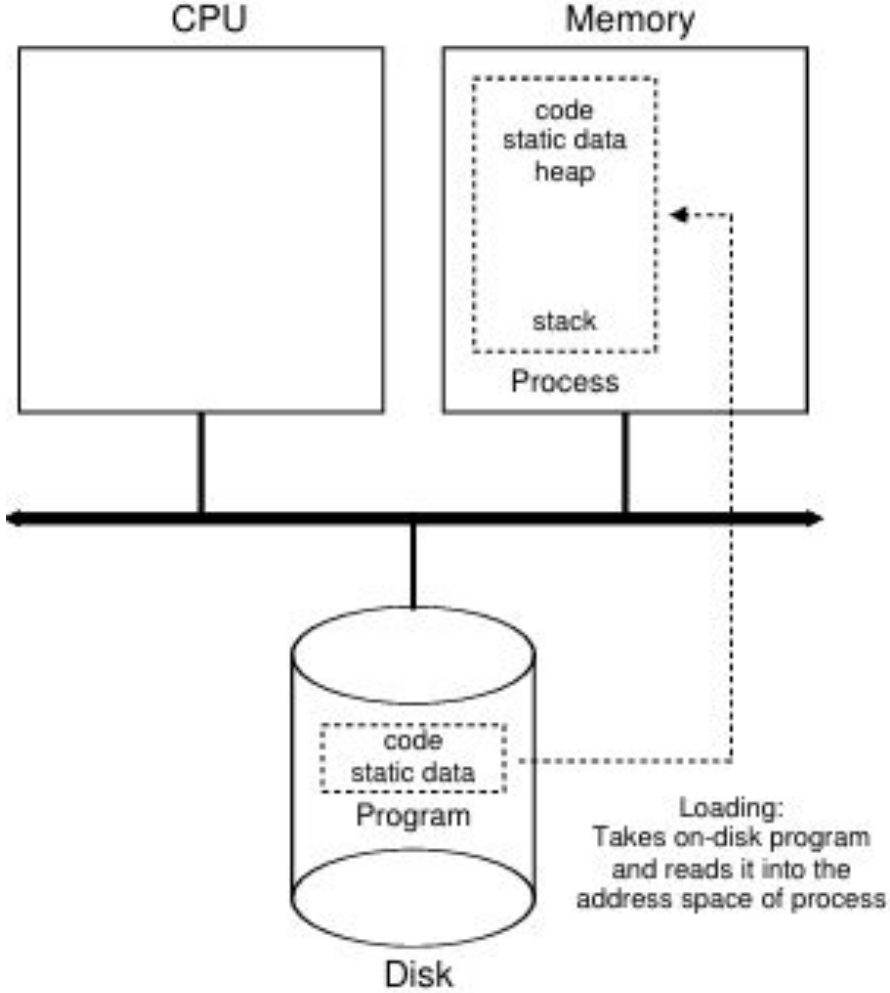
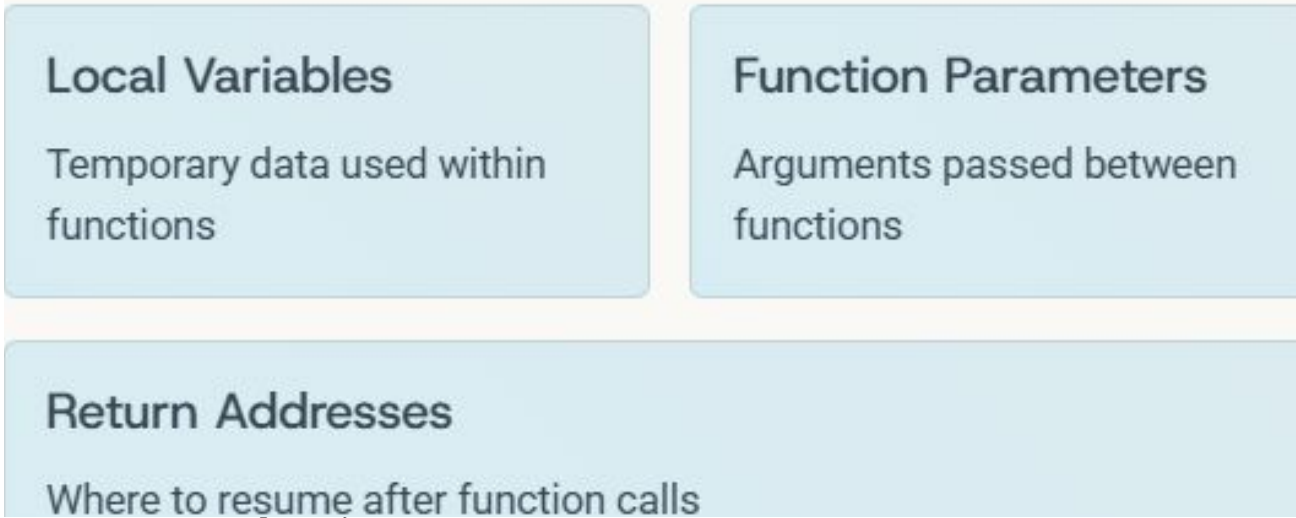


Figure 4.1: **Loading: From Program To Process**

2.Allocating Stack Memory

- The OS creates a **runtime stack** for the process.
- The stack holds:
  - Local variables
  - Function parameters
  - Return addresses

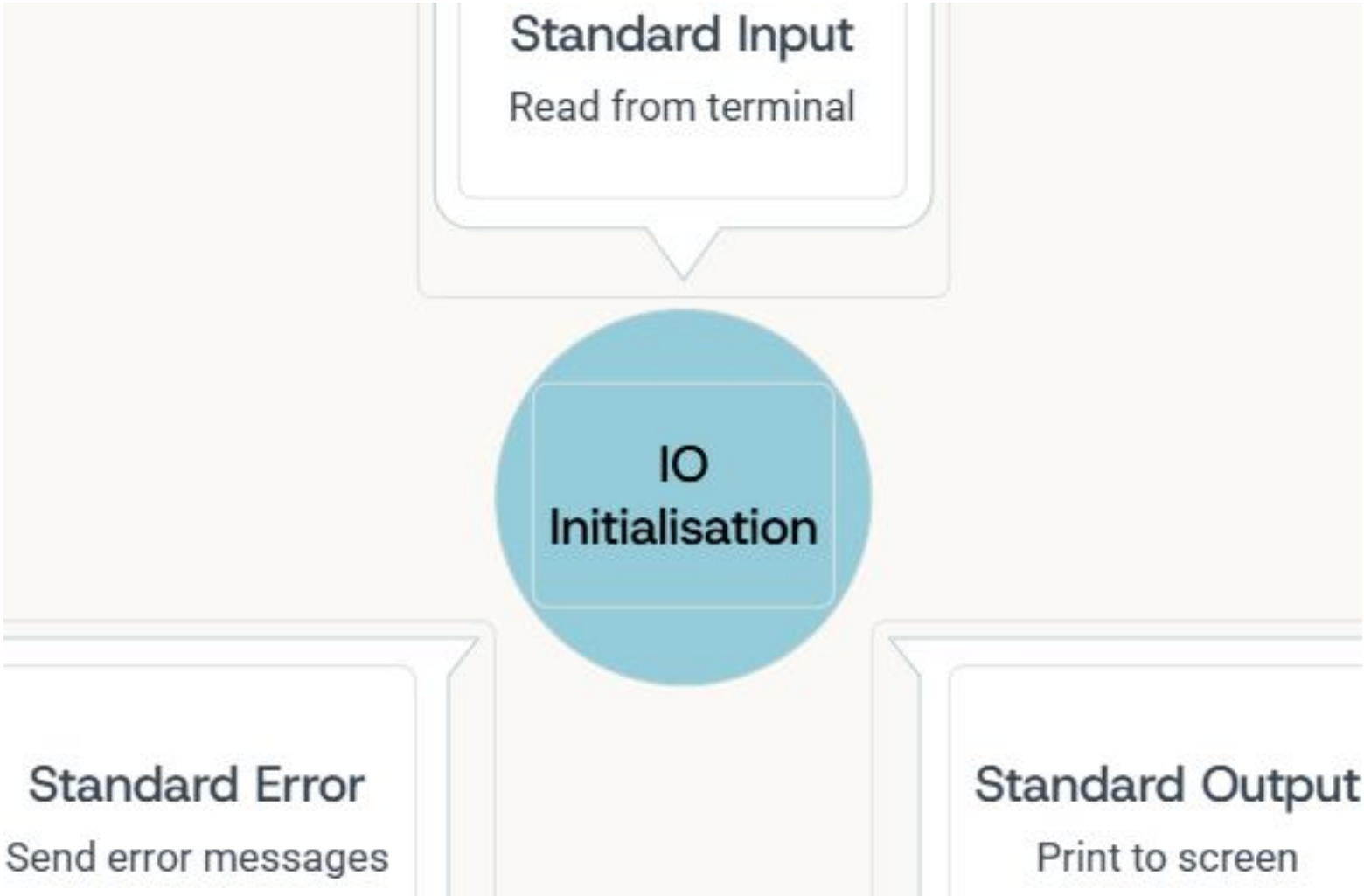


3. Allocating Heap Memory

- The OS prepares an empty area for the **heap**.
- The heap is used for dynamic memory allocation through functions such as:
  - malloc()
  - free()
- It starts small and grows as needed.

4. Initializing I/O Resources

- Every process in UNIX-like systems starts with three default file descriptors.
- These file descriptors provide basic input and output functionality.
- **Standard Input:** Used to receive input, typically from the keyboard.
- **Standard Output :** Used to display normal output on the screen.
- **Standard Error :** Used to display error messages separately.

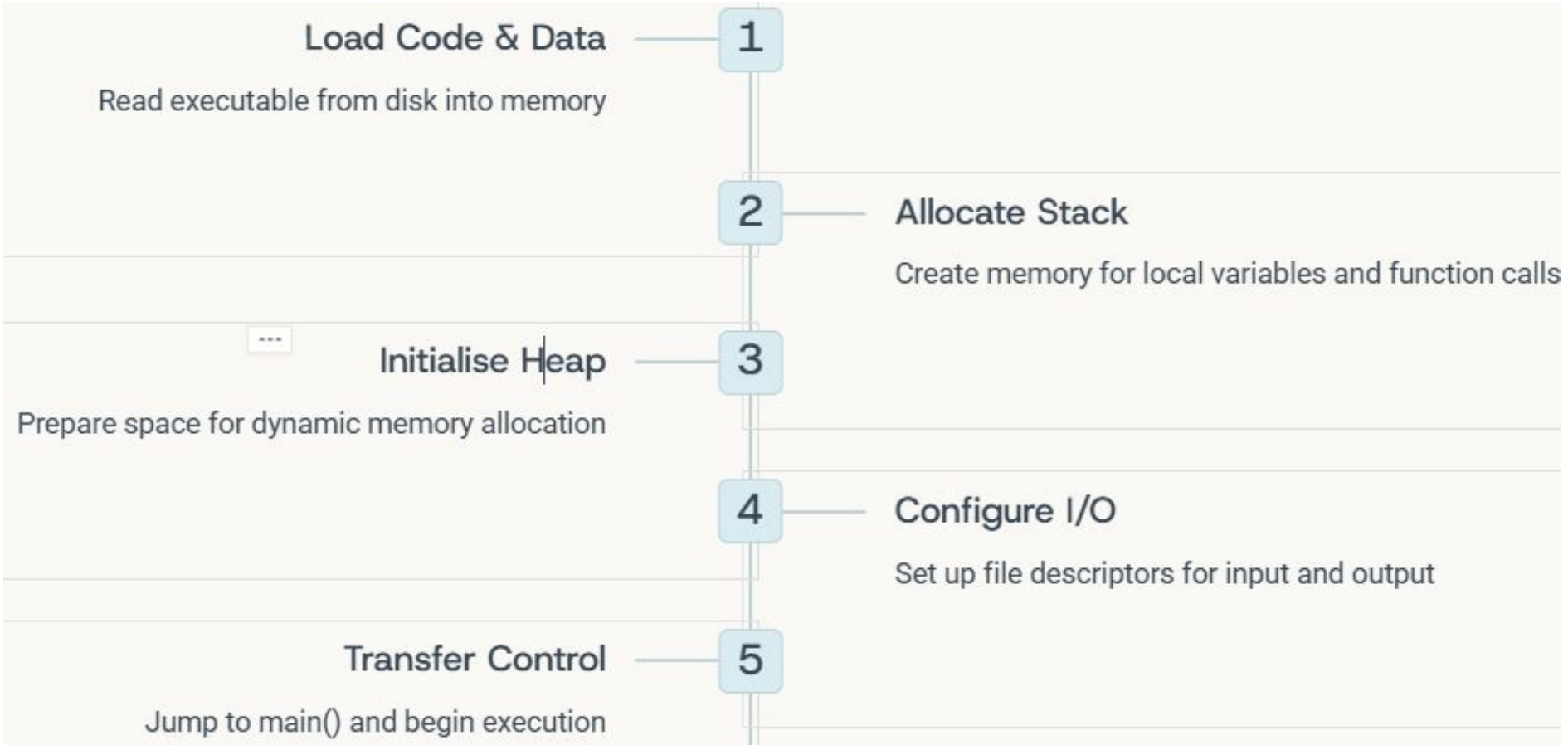

Standard Input
Read from terminal
IO
Initialisation
Standard Error
Send error messages
Standard Output
Print to screen

5. Starting Execution

Once memory and I/O setup is complete, the OS:

- Sets the program's **entry point** (usually the main() function).
- Loads the process's initial CPU state (registers, PC, stack pointer).
- **Jumps** to main().
- CPU control is now transferred to the new process.

At this moment, the program officially begins execution.


Load Code & Data
1
Read executable from disk into memory
2
Allocate Stack
Create memory for local variables and function calls
Initialise Heap
3
Prepare space for dynamic memory allocation
4
Configure I/O
Set up file descriptors for input and output
Transfer Control
5
Jump to main() and begin execution

Process	Program
An instance of a program being executed.	A collection of instructions written to perform a specific task for the user.
Short lifetime	Long lifetime
Requires resources such as memory, CPU, I/O devices, etc...	Stored in hard-disk and doesn't requires any resources
Has a dynamic instance of code and data	Has static code and data.
Running instance of code	Executable code

Process States

Prepared by Dilsha K P

Process states

★ When a process executes, it passes through different states.

★ A process state is a condition of the process at a specific instant of time.

★ It also defines the current position of the process.

1. **new**:

The process has been created but has not yet been scheduled for execution

2. **ready**:

process is ready to be assigned to a processor and waits for the OS to allocate the processor to it.

3. **running**:

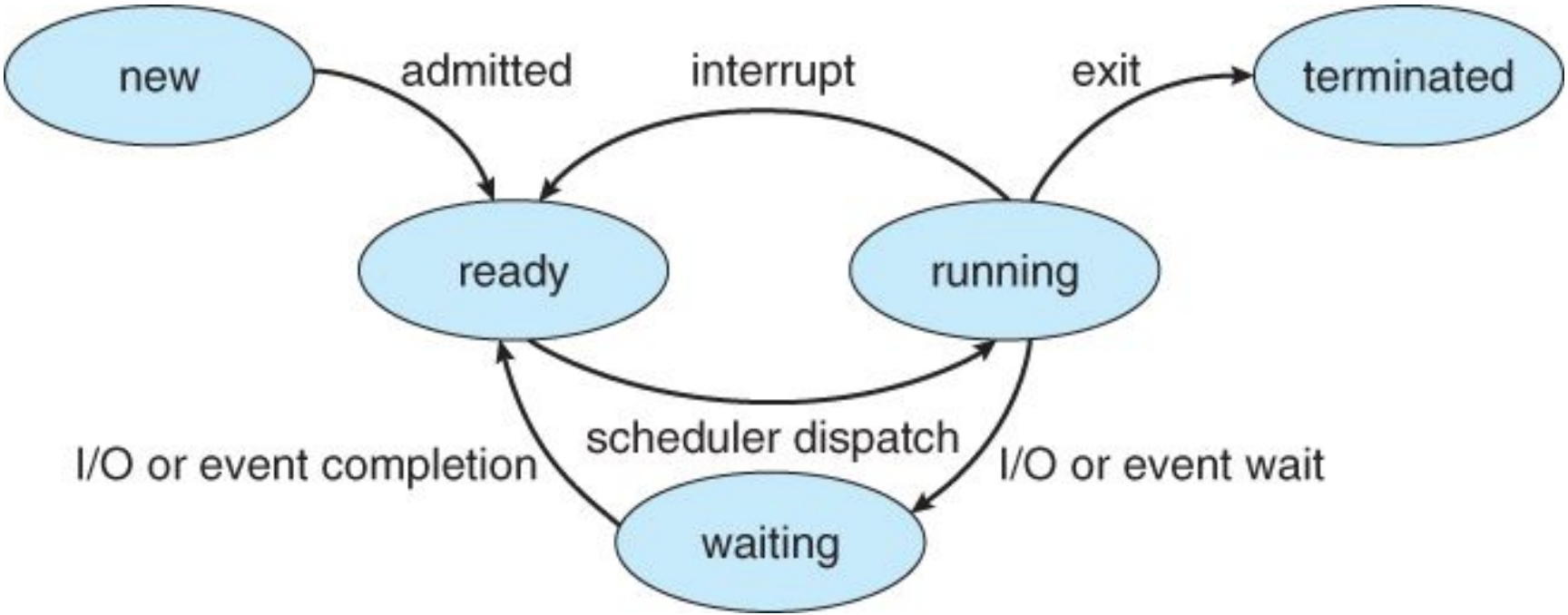
After being assigned to a processor, the process enters into the running state and the processor executes its instructions.

4. waiting:

- The process is waiting for an event to occur, such as the completion of an I/O operation.
- Moves back to **Ready** state once the event is completed.

5. terminated:

- The process has finished execution.
- OS deallocates resources associated with the process.


new
admitted
interrupt
exit
terminated
ready
running
scheduler dispatch
I/O or event completion
I/O or event wait
waiting

Three Process States

Running

The process is executing instructions on a processor at this moment.

Ready

The process is prepared to run, but the OS has chosen not to execute it right now.

Blocked

The process has performed an operation requiring it to wait for an event before continuing.

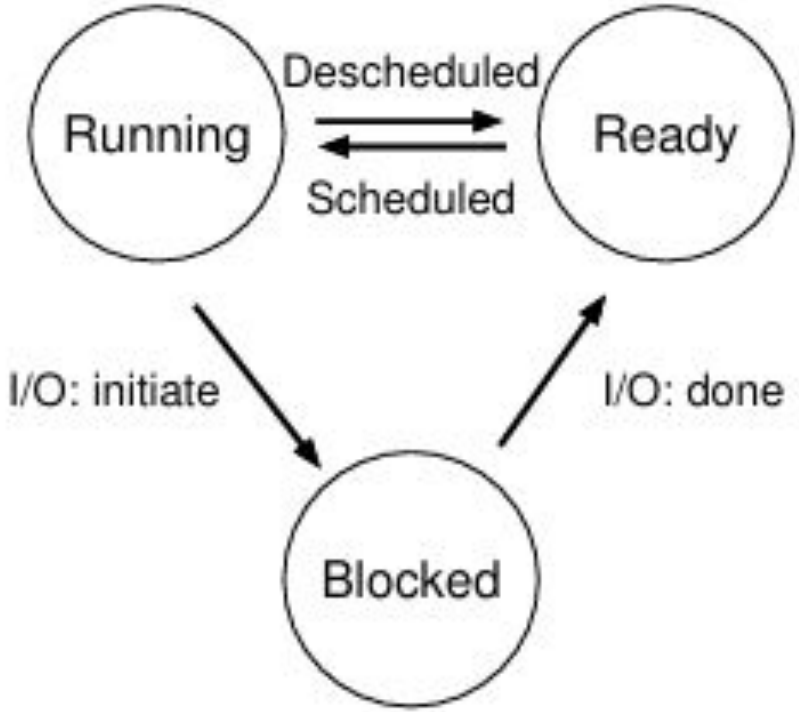


Figure 4.2: **Process: State Transitions**

Running:

In the running state, a process is running on a processor. This means it is executing instructions.

2. Ready

- The process is prepared to run **but is not currently executing**.
- It is waiting for the OS **scheduler** to assign CPU time.
- Processes in the ready state are usually placed in a **ready queue**.

Blocked State

Waiting for Events

A process enters the blocked state when it performs an operation requiring external completion, such as disk I/O or network requests.

Common Example

When a process initiates an I/O request to a disk, it becomes blocked, allowing other processes to use the processor whilst waiting.

- The process cannot continue until an event happens.
- Example: waiting for input/output (I/O) operation like reading from disk.

- A process enters the **Blocked** state when it performs an operation that prevents it from continuing execution **until some specific event occurs**.
- In this state, the process **cannot use the CPU**, even if the CPU is free, because it is waiting for something to finish.

Example – I/O Request

A common situation is when a process requests I/O, such as reading data from a disk.

Disk operations are much slower than CPU operations. So:

1. The process asks the disk to read data.
2. It **cannot continue** until the data arrives.
3. The operating system marks the process as **Blocked**.
4. While it is blocked, the CPU is given to **another process** that is ready to run.

This ensures the CPU is not wasted while waiting for slow operations to complete.

If two processes only use the CPU (no I/O), their states over time could look like this:

Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	$Process_0$ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	$Process_1$ now done

If Process0 does I/O, the state transitions look different:

Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	$Process_0$ initiates I/O
4	Blocked	Running	$Process_0$ is blocked,
5	Blocked	Running	so $Process_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	$Process_1$ now done
9	Running	–	
10	Running	–	$Process_0$ now done

Figure 4.4: **Tracing Process State: CPU and I/O**

State Transitions

- **Ready → Running:**
  The OS decides to give the CPU to the process. This is called **scheduling**.

- **Running → Ready:**
  The OS stops the process (maybe because time is up) and makes it wait again. This is called **descheduling**.

- **Running → Blocked:**
  The process requests something like **I/O (e.g., disk read)** and cannot continue until it is completed. So, it becomes **blocked**.

- **Blocked → Ready:**
  After the requested event (like I/O completion) happens, the process becomes **ready again** to run.

- Once ready, the OS may immediately run it or keep it waiting depending on scheduling.

Scheduling & Descheduling

Scheduled

The OS moves a process from ready to running, allocating processor time.

Descheduled

The OS moves a process from running back to ready, freeing the processor for others.

Data Structures in Operating Systems

OS track information about processes using sophisticated data structures.

Understanding these structures reveals how OSes manage multiple programs simultaneously.

Prepared by Dilsha K P

Process list

The operating system maintains a **process list** to keep track of all active processes in the system.

This list acts as a central data structure where the OS stores essential information about each process such as its state, memory usage, CPU registers.

Process Control Block (PCB): A structure containing all information about a single process.

Why is the Process List Important?

The process list helps the OS manage multitasking efficiently. It keeps track of:

- **Processes ready to run**
- **Processes currently running**
- **Processes that are blocked or waiting (e.g., for I/O operations)**

This allows the scheduler to decide **which process should get the CPU next**.

When a process performs an I/O operation (like reading from a file or disk), it cannot continue executing immediately. So:

1. The OS marks the process as **blocked** and stores it in the process list.

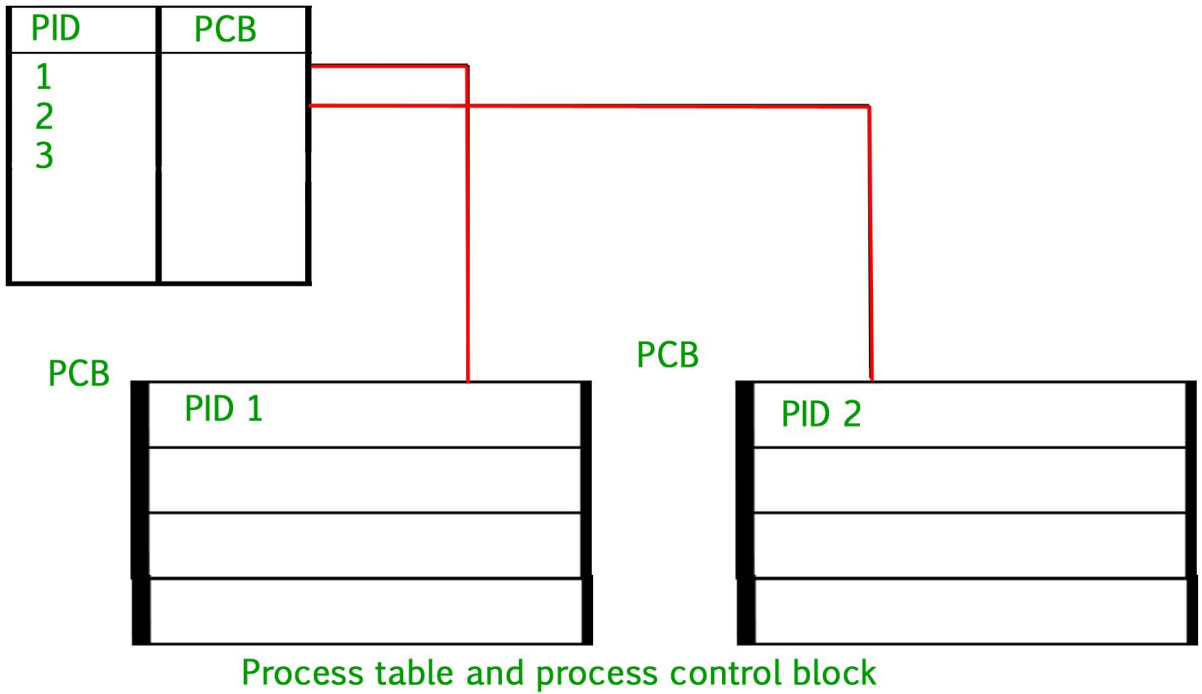
2. The CPU switches to another runnable process.

3. When the I/O operation finishes, the OS **identifies the blocked process in the process list**.

4. The OS changes its state to **RUNNABLE**, allowing the scheduler to execute it again.

PCB = Information about one process

Process List = Collection of PCBs for ALL processes



Process table and process control block

- The **process list** keeps track of all processes that are currently active in the system.
- Each entry in the process list stores details about a single process.
- This individual structure is known as a **Process Control Block (PCB)**.
- A PCB is simply a data structure that contains essential information about a process such as its ID, state, memory usage etc.

The xv6 Process Structure

The xv6 kernel demonstrates how operating systems track process information. Similar structures exist in Linux, macOS, and Windows, though they are considerably more complex.

Register Context

Stores register state when a process stops, enabling the OS to resume execution later.

Process State

Tracks whether the process is running, ready, blocked, or in another state.

Memory Information

Records process memory location and size for proper resource allocation.

The register context stores the contents of a process's CPU registers when the process is stopped.

When a process is paused, its register state is saved in this memory location.

Later, the operating system can resume the process by restoring these saved values back into the physical CPU register

Figure 4.3 shows what type of information an OS needs to track about each process in the xv6 kernel

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
  int eip;
  int esp;
  int ebx;
  int ecx;
  int edx;
  int esi;
  int edi;
  int ebp;
};
```

eip – Instruction pointer

esp – Stack pointer

ebx, ecx, edx – General purpose

esi, edi – Index registers

ebp – Base pointer

```
// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```

- **UNUSED**

- No process assigned to this slot.

- **EMBRYO**

- Newly created process.

- Memory and PCB are being initialized.

- **SLEEPING**
  - Process is blocked and waiting for an event (e.g., I/O completion).
- **RUNNABLE**
  - Ready to run but waiting for CPU time from scheduler.
- **RUNNING**
  - Process is currently executing on the CPU.
- **ZOMBIE**
  - Process has finished execution.
  - Resources not yet released (waiting for parent cleanup).

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
  char *mem;                  // Start of process memory
  uint sz;                    // Size of process memory
  char *kstack;               // Bottom of kernel stack
                              // for this process
  enum proc_state state;      // Process state
  int pid;                    // Process ID
  struct proc *parent;        // Parent process
  void *chan;                 // If non-zero, sleeping on chan
  int killed;                 // If non-zero, have been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;          // Current directory
  struct context context;     // Switch here to run process
  struct trapframe *tf;       // Trap frame for the
                              // current interrupt
};
```

Zombie State

- A process can be in several possible states. Beyond simple running, ready, or blocked, there may be:
- An initial state when the process is being created.
- A final state after the process has finished execution but has not been fully removed from the system.
- In UNIX-based systems, this is called the **zombie state.**

Zombie State

- A zombie process is one that has completed execution but its PCB still exists.
- Purpose: allows the parent process to read the exit status (success/failure).
- The parent uses the system call wait() to:
  - Collect the child's exit code
  - Inform OS to remove the PCB and free resources

What is Context Switching?

- A context switch is the process of storing the state of a currently running process and loading the state of another process so that the CPU can execute it.

- Since the CPU can run only one process at a time, the operating system switches between processes very quickly, giving the illusion of parallelism.

Why Context Switching is Needed?

- Context switching allows:
  - Multitasking (multiple processes share the CPU)
  - Responsiveness (OS can switch to important tasks)
  - Fairness (no process hogs the CPU)
  - Handling interrupts (e.g., I/O, timers)

Process API

Prepared by Dilsha K P

Process API

A Process API is a set of functions provided by the operating system to manage processes.

1. **Create**
   - OS must provide a way to create new processes.
   - Examples:
     - Typing a command in terminal
     - Double-clicking an application icon
   - The OS loads the program into memory, sets up its PCB, and starts execution.
   - In UNIX systems, this is done using fork() + exec().

2. Destroy

- Processes may need to be stopped forcefully.
- Used when:
  - A program hangs
  - A program misbehaves (infinite loop, crash)
- OS provides system calls to terminate or kill a process.
- Example in UNIX: kill(pid)

3. Wait

- A process may need to wait for another process to finish.
- Very common in parent-child relationships.
- Parent uses the wait system call to:
  - Pause until the child terminates
  - Collect the child's exit code
- UNIX example: wait() or waitpid().

Miscellaneous Control

The operating system also offers additional ways to control processes apart from creating, destroying, or waiting for them.

For example:

- **Suspend and Resume:**
  The OS can temporarily pause a process and continue it later. This helps manage CPU usage and system responsiveness.

- **Change Priority:**
  The OS can increase or decrease a process's priority. A higher priority process receives more CPU time, while a lower priority process gets less.

- **5. Status**
- OS provides interfaces to get information about a process such as:
  - Current state (running, ready, blocked, zombie)
  - CPU time consumed
  - Memory usage
  - Process ID, parent ID
  - Exit status (after termination)

fork() System Call

- Definition
  - fork() is a system call used in UNIX/Linux to create a new process.
  - The new process created is called the child process.
  - The process that called fork() is the parent process.
- The child process is an almost exact copy of the parent:
  - Same code
  - Same memory layout initially
  - Own copy of registers, PC, and variables

What is the Use of Fork System Call?

- The use of the fork() system call is to create a new process by duplicating the calling process.
- The fork() system call is made by the parent process, and if it is successful, a child process is created.

The fork() system call does not accept any parameters.

It simply creates a child process and returns the process ID.

If a fork() call is successful :

- The OS will create two identical copies of the address space for parent and child processes. Therefore, the address spaces of the parent and child processes are different.
- The new child process's process ID, or PID, is returned to the parent process. If something goes wrong, the parent process returns -1.
- Zero is returned to the new child process. (In the case of failure, the child process is not created).

Case 1: fork() returns -1 (Failure)

The child process is not created.

This happens due to errors like lack of system resources.

The return value -1 is received by the parent process.

👉 Meaning: Process creation failed.

Case 2: fork() returns 0 (Child Process)

- The return value **0** is received in the **child process**.
- The child process is a **copy of the parent process**.
- The child gets a **new PID**.

👉 *Meaning:* This code is executed by the child.

Case 3: fork( ) returns a positive value (Parent Process)

- The return value is the **PID of the child process**.
- This value is received by the **parent process**.

👉 *Meaning:* This code is executed by the parent.

- Both processes continue execution from the next line after fork().
- Return Values
  - fork() returns 0 to the child.
  - fork() returns the child's PID to the parent.
  - If fork fails, it returns -1.

EXAMPLE 1

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

//main function begins
int main()
{
    fork();
    print("the process is created using fork() system call\n");
    return 0;
}
// Using fork() system call single time
```

Output:

```
the process is created using fork() system call
the process is created using fork() system call
```

How this program works

1. The program starts as **one parent process**.
2. fork() is executed:
    - A **child process is created**.
    - Now there are **two processes**: parent and child.
3. Both the **parent and the child continue execution from the next line**.
4. Both processes execute
5. Therefore, **printed two times**.

wait() System Call

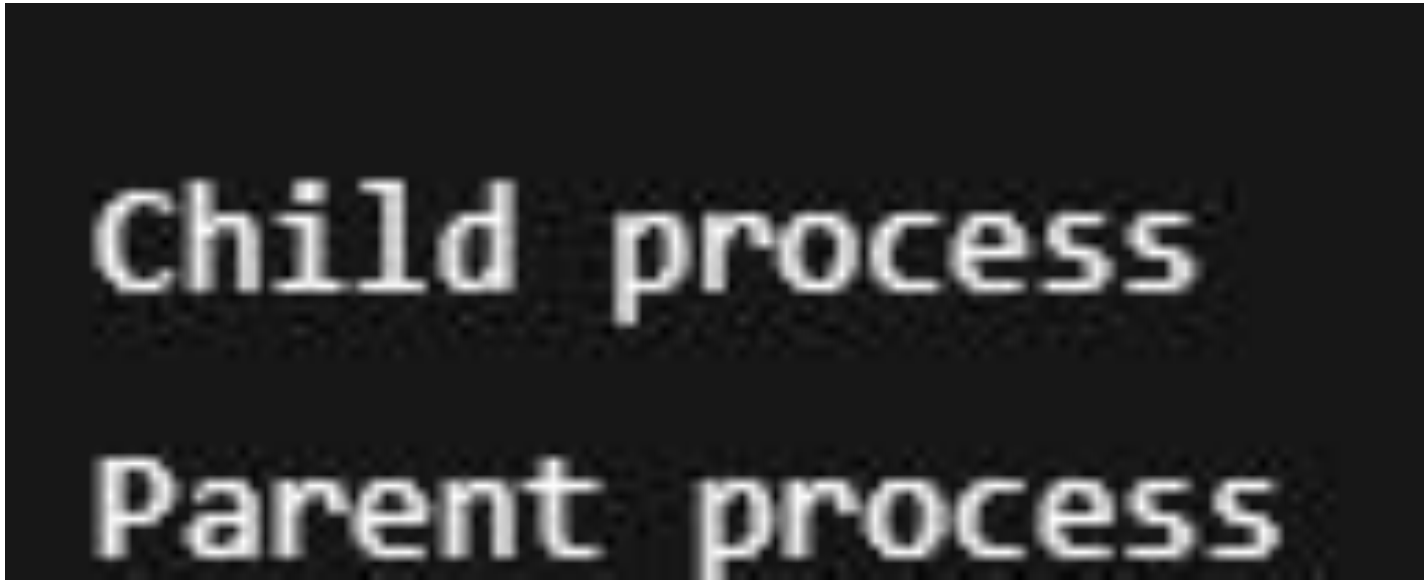
- Definition
  - wait() makes the parent process wait until its child finishes.
  - Prevents the parent from running before the child completes.
- Effect
  - Makes the program output deterministic.
  - Useful for synchronization between parent and child.

How it works:

- The parent calls wait() after fork().
- The parent pauses until the child finishes execution.

Effect: Makes execution **deterministic**, ensuring the child runs first.

```c
int main() {
    if (fork() == 0) {
        printf("Child process\n");
    } else {
        wait(NULL);
        printf("Parent process\n");
    }
    return 0;
}
```


Child process
Parent process

Step 1: Program starts

- Only **one process** exists → the **parent process**.

Step 2: `fork()` is called

- A **child process is created**.
- Now there are **two processes**:
    - **Child**
    - **Parent**

Step 3: Return value of `fork()`

- In the **child process** → `fork()` returns **0**
- In the **parent process** → `fork()` returns **positive value** (child's PID)

Step 4: `if-else` decision

- **Child process (`fork() == 0`)**

Executes:

```
printf("Child process\n");
```

  -
  - Child finishes execution.

- **Parent process (`fork() > 0`)**

Executes:

```
wait(NULL);
```

  -
  - Parent **waits until the child finishes**

After child ends, parent prints:

```
printf("Parent process\n");
```

  - **Step 5: Program ends**
- Child ends first
- Parent ends after child

exec ( ) System Call

- The exec system call is used by a process to replace its current memory image with a new program.
- In simpler terms, it loads and runs a different program within the same process.

Purpose:

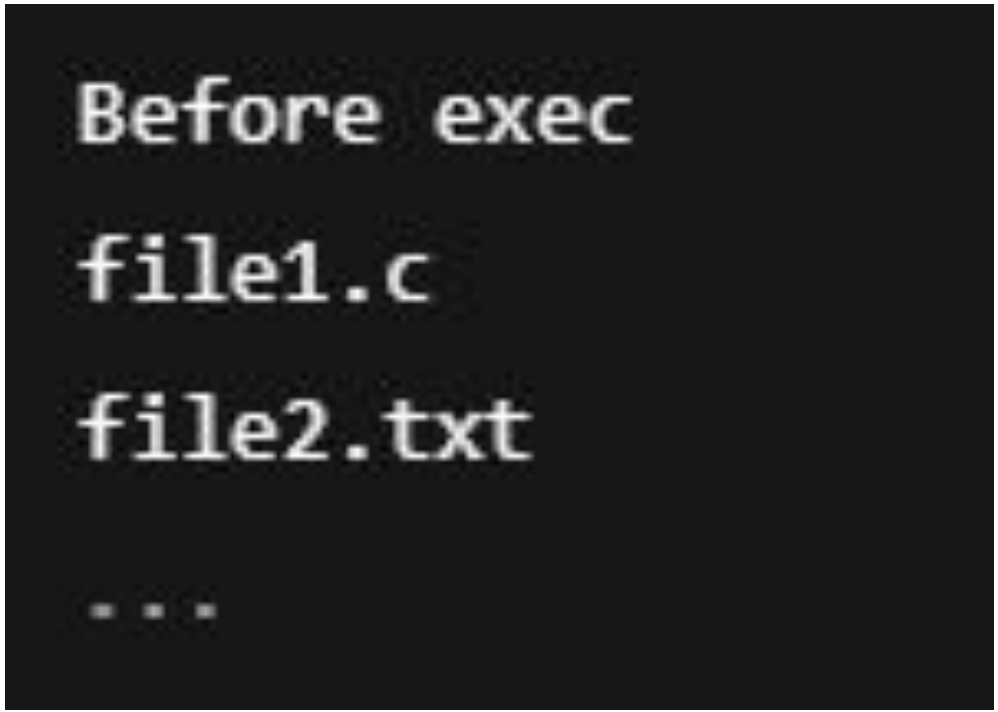
- To run a new program in the context of the current process.
- The process ID (PID) remains the same, but the previous code, data, and stack are replaced.

1. `exec()` replaces the **current process image** with a **new program**.
2. It does **not create a new process**; the process calling `exec()` becomes the new program.
3. Typically combined with `fork()`:
    - `fork()` creates a child process.
    - The child process calls `exec()` to run a **different program**.
    - The parent can use `wait()` to wait for the child to finish.
4. Example use case: Run `wc` (word count) on a file.

```c
int main() {
    printf("Before exec\n");

    char *args[] = {"ls", NULL};
    execvp(args[0], args);  // Replace this program with ls

    // This line will not execute if execvp succeeds
    printf("After exec\n");

    return 0;
}
```

Explanation:

1. `printf("Before exec")` runs first.
2. `execvp("ls", args)` replaces the current program with `ls`.
3. The program stops running itself; the output of `ls` is shown.
4. `printf("After exec")` **never runs**.


Before exec
file1.c
file2.txt
...

Sharing processor among processes

Prepared by Dilsha K P

A computer usually has **one physical CPU**, but many programs (processes) want to run at the same time.

The **operating system (OS)** gives each process a **small slice of CPU time** called a **time slice** or **quantum**.

After a process's time slice ends, the OS **pauses it** and switches to the next process.

This switching happens **very fast**, often millions of times per second.

- Even though only **one process runs at a time**, to a user it looks like **all processes are running simultaneously**.

- This is called **virtualizing the CPU,** because the OS makes it seem like each process has its own CPU.

CPU Virtualization

1. A CPU can execute **only one instruction at a time**.
2. The OS quickly **switches between multiple processes**, giving each a short turn on the CPU.
3. Because the switching is very fast, **each process seems to run at the same time**.
4. **CPU virtualization:** This creates the **illusion that each process has its own CPU**, even though there's only one physical CPU.

User Mode and Kernel Mode

- Modern CPUs support two execution modes to ensure safety and controlled access to hardware:

1.User Mode

- Normal applications run here.
- Restricted access → cannot perform privileged operations (I/O, device access, change trap tables, etc.).
- If a user process attempts a privileged operation → hardware raises an exception → OS handles it.

2. Kernel Mode

- Operating System runs here.
- Has full access to hardware (disk, memory, CPU settings, interrupts, trap table).
- Performs tasks such as scheduling, memory management, device control.

Refer Page no: 40

System Calls

- Mechanism for user programs to safely request privileged operations.
- Achieved through a trap instruction → CPU switches to kernel mode → executes correct kernel function → returns using return-from-trap to user mode.

Refer page no : 42

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process Control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File Manipulation	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device Manipulation	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	WriteConsole()	write()
Information Maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communication	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shmget()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitlializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`

Context Switching

A **context switch** happens when the operating system stops one process and starts or resumes another.

This allows multiple processes to share the CPU efficiently.

When Context Switching Happens

1. **Timer Interrupt:**
    - In **preemptive multitasking**, the OS uses a timer to periodically switch processes.
2. **Voluntary Yield:**
    - In **cooperative multitasking**, a process may voluntarily give up the CPU.
3. **Blocking I/O:**
    - When a process waits for input/output (like reading a file), it can't continue, so the OS switches to another process.
4. **Higher-Priority Process:**
    - If a more important process needs the CPU, the OS can preempt the current process.

1. Timer Interrupt

- In **preemptive multitasking**, the operating system uses a **timer**.
- When the timer expires, an **interrupt occurs**.
- The OS **stops the running process** and switches the CPU to another process.
- This allows **fair CPU sharing** among processes.

👉 *Example:* Time-sharing systems like Linux, Windows.

Voluntary yield means:

👉 A **running process willingly stops using the CPU** and allows another process to run.

- The process **decides by itself**:
  "I don't need the CPU now."

- It **hands over the CPU** to the operating system.

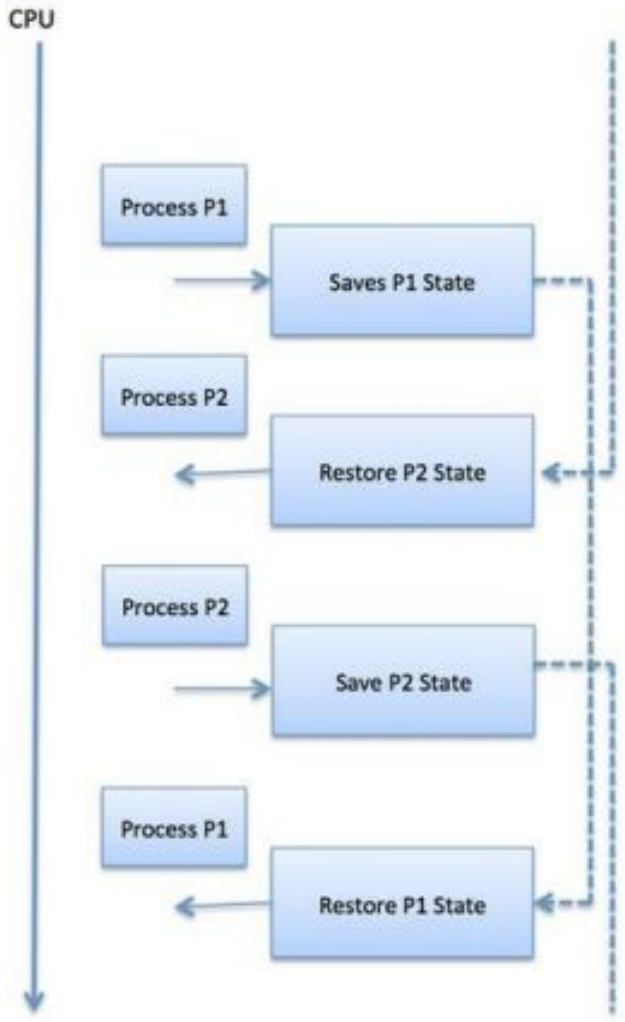
- The OS then **switches to another process**.

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process.

State Save

State Restore

This task is known as a **context switch**.


CPU
Process P1
Saves P1 State
Process P2
Restore P2 State
Process P2
Save P2 State
Process P1
Restore P1 State

Steps in Context Switching

1. **Interrupt or System Call:**
    - A timer interrupt or system call occurs, causing the CPU to **switch from user mode to kernel mode**.
2. **Save Current Process Context:**
    - CPU state is saved so the process can resume later:
        - CPU registers
        - Program Counter (PC)
        - Stack Pointer (SP)
    - Stored in the **Process Control Block (PCB)** or kernel stack.

3. Load Next Process Context:

- OS **loads the saved information** of the next process.
- This includes **CPU registers and the Program Counter (PC)**.
  Now the CPU is ready to run the **next process**.

4. Return to User Mode:

- After loading the process details, the CPU **switches from kernel mode to user mode**.

- The next process **continues execution from the exact instruction where it stopped earlier**.

System boot sequence

System Boot Sequence

The **system boot sequence** is the process by which a computer starts up and loads the operating system so it becomes ready to use.

1. Bootstrap / Boot Loader Execution

When the computer is **powered ON**, the firmware (**BIOS or UEFI**) starts running.

It executes a small program called the **bootstrap program** or **boot loader**.

The **main job** of the boot loader is to:

- **Find the operating system kernel** on storage (hard disk/SSD)
- **Load the kernel into main memory (RAM)**

2. Loading the Kernel

- The boot loader finds the **kernel image** (e.g., vmlinuz in Linux).
- Loads it into **main memory**.
- If compressed, the kernel is **decompressed** after loading.

3. Kernel Startup

- The kernel begins execution.
- Initializes important components:
  - CPU registers
  - Memory management
  - Device controllers and other hardware

4. Mounting the Root File System

- A temporary **RAM file system (initramfs)** is created to hold essential drivers for disk access.

- After drivers are loaded, the kernel switches to the **actual root file system**.

5. Starting the Initial Process

- In Linux, the kernel starts **systemd (PID 1)**.
- systemd launches background services (networking, databases, web servers).
- Finally, the **login prompt** or graphical interface is presented.

Boot Methods: BIOS vs UEFI

- **BIOS-Based Boot**: This uses a multistage boot process.
- BIOS loads a small boot loader from the boot block, and that boot loader loads the full OS.
- **UEFI Boot**: This is a modern replacement for BIOS that acts as a complete boot manager.
- It is faster and supports 64-bit systems and large disks.

*UEFI Boot (Unified Extensible Firmware Interface)

BIOS-Based Boot

- Uses a **multi-stage boot process**.
- **BIOS** loads a **small boot loader** from the **boot sector (boot block)**.
- The boot loader then **loads the full operating system**.
- It is **older and slower**.
- Limited support for **large disks**.

UEFI Boot

- **UEFI (Unified Extensible Firmware Interface)** is a **modern replacement** for BIOS.
- Works as a **complete boot manager**.
- **Faster booting** compared to BIOS.
- Supports **64-bit systems** and **very large disks**.
- Commonly used in **modern computers**.

GRUB Boot Loader (Linux/UNIX)

- Open-source boot loader.
- Reads configuration files at startup.
- Allows selecting different kernels and modifying boot parameters.

Threads and Concurrency

Prepared by Dilsha K P

Thread

In Operating Systems, **a thread** is the smallest unit of execution within a process.

Components of a Thread

Each thread has its own:

- Thread ID
- **Program Counter (PC)** – points to the current instruction being executed
- **Register set** – stores temporary data and CPU state
- **Stack** – stores local variables, function calls, and return addresses

Threads belonging to the **same process share**:

- **Code section**
- **Data section**
- **Open files**
- **Operating system resources (signals, memory, etc.)**

Threads are also termed lightweight processes as they share common resources.

Eg: While playing a movie on a device the audio and video are controlled by different threads in the background.

Single-threaded vs Multithreaded Process

Single-threaded Process

- Only one thread executes at a time
- One task must finish before another begins
- If the thread blocks (e.g., I/O operation), the whole process waits

Multithreaded Process

- Multiple threads execute concurrently
- Can perform multiple tasks at the same time
- If one thread blocks, other threads can continue execution

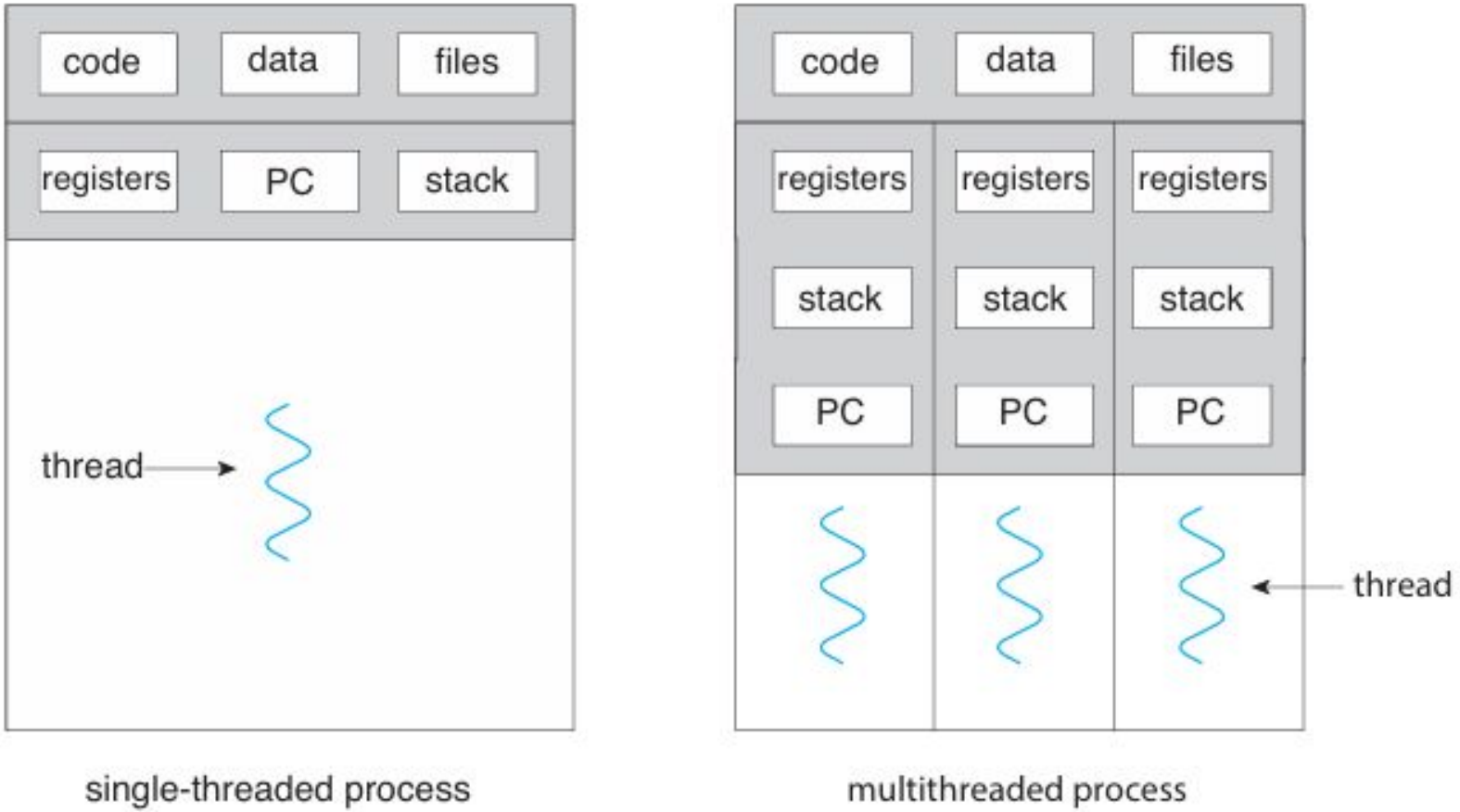


Figure 4.1 Single-threaded and multithreaded processes.

Aspect	Single Thread	Multi-Thread
Definition	A program executes with one thread of control.	A program executes with multiple threads of control.
Performance	Slower as tasks are executed sequentially.	Faster as multiple tasks can run concurrently.
Resource Sharing	No sharing of resources between threads.	Threads share the process's memory and resources.
Responsiveness	Less responsive during long operations.	More responsive as threads can handle tasks simultaneously.
Scalability	Limited to a single processing core.	Can utilize multiple cores for parallel execution.
Use Case	Suitable for simple, sequential applications.	Suitable for complex, interactive, or parallel tasks.

Why Threads?

- Most modern apps are multithreaded:
  - Browser → one thread loads data, another displays content
  - Word processor → typing, spell check, graphics each in different threads
  - Photo app → each thumbnail created by a separate thread
  - Web server → each client request handled by a separate thread

Multithreading

- **Multithreading** refers to the ability of a CPU or an OS to manage multiple threads of execution within a single process.
- The idea is to achieve parallelism by dividing a process into multiple threads.
- Each thread represents a separate flow of control, enabling tasks to run concurrently.
- Multiple threads execute simultaneously, improving performance by utilizing CPU cores efficiently
- Threads in a process share the same address space, global variables, and system resources

- A process can have multiple threads.
- Each thread will have their own task and own path of execution in a process.
- For example, in a browser, multiple tabs can be different threads.
- MS Word uses multiple threads: one thread to format the text, another thread to process inputs, etc.
- Multithreading is a technique used in operating systems to improve the performance and responsiveness of computer systems.

Multithreading benefits

he benefits of multithreaded programming can be classified into **four major categories**:

1. Responsiveness

- Multithreading keeps an application responsive even if part of it is blocked or performing a long operation.
- Especially important for **interactive applications** and **user interfaces**.

Example:

- When a user clicks a button that starts a time-consuming task:
  - In a **single-threaded application**, the application becomes unresponsive until the task finishes.
  - In a **multithreaded application**, the long task runs in a separate thread, allowing the application to continue responding to user actions.

2. Resource Sharing

- Processes can share resources only through explicit mechanisms like **shared memory** or **message passing**.
- Threads automatically share the **memory and resources of the same process**.

Benefits:

- Shared code and data
- Efficient communication between threads
- Multiple threads can work within the same address space

3. Economy

- Creating a process requires allocating separate memory and resources, which is **costly**.
- Thread creation is **less expensive** because threads share the process resources.

Key points:

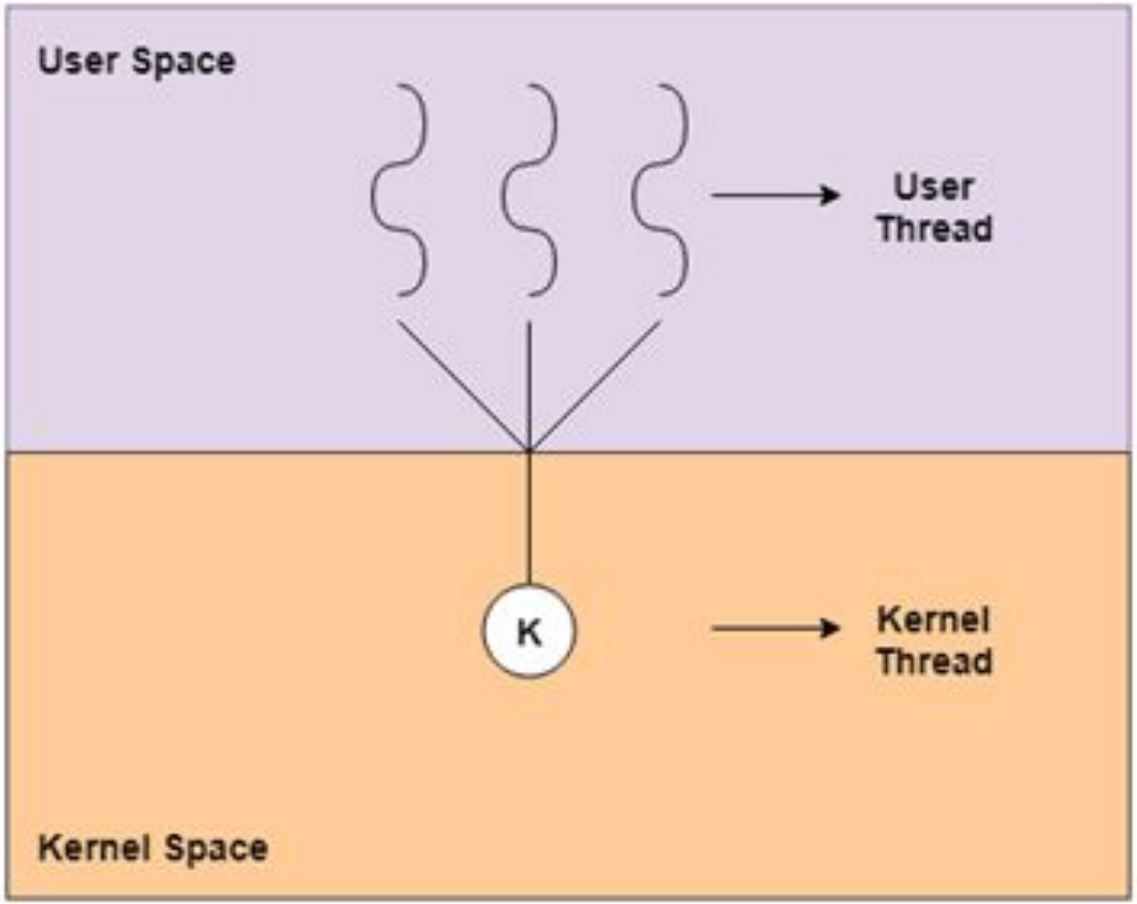
- Less memory consumption
- Faster thread creation compared to process creation
- Context switching between threads is faster than between processes

4. Scalability

- Multithreading is highly beneficial on **multiprocessor and multicore systems**.
- Different threads can execute in parallel on different CPU cores.
-

Types of Threads

- User Level Thread
- Kernel Level Thread


User Space
User
Thread
K
Kernel
Thread
Kernel Space

USER LEVEL THREADS	KERNEL LEVEL THREADS
Created by user at user space using thead library	Created by kernel at kernel space
Easy to create and manage & faster	Slow to create and manage
Less time for context switching Not require h/w support	More time for context switching require h/w support
OS does not recognise user level threads	Kernel level threads are recognised by OS
One user level thread perform blocking operation Then entire process will be blocked	One Kernel level thread perform blocking operation No effect on another Kernel level thread
Run on any OS	Specific to OS

Multithreading models

Prepared by Dilsha K P

Multithreading

- Multithreading is a programming and execution model that allows multiple threads to exist within a single process, executing concurrently.
- Each thread represents a separate path of execution, enabling better responsiveness, resource utilization and parallelism, especially in modern multiprocessor systems.
- *Multithreading is widely used in applications like web servers, interactive applications and high-performance computing where concurrent execution is essential.*

Multithreading models

Threads can be managed either by the **user** or by the **operating system (kernel)**.

- **User threads** are created and managed by a thread library in user space, without kernel support.

- **Kernel threads** are created and managed directly by the operating system.

Modern operating systems such as **Windows, Linux, and macOS** support **kernel threads**.

To execute threads efficiently, there must be a relationship between **user threads** and **kernel threads**. This relationship is defined by **multithreading models**. The three common models are:

1. **Many-to-One Model**

2. **One-to-One Model**

3. **Many-to-Many Model**

1. Many-to-One Model

- **Many user threads are mapped to a single kernel thread**.
- Thread management is done in **user space**, so it is fast and efficient.
- **Disadvantages:**
  - If one thread makes a **blocking system call**, the entire process blocks.
  - Only one thread can run at a time, even on multicore systems.
- This model **cannot achieve parallelism**.
- This model is rarely used today.

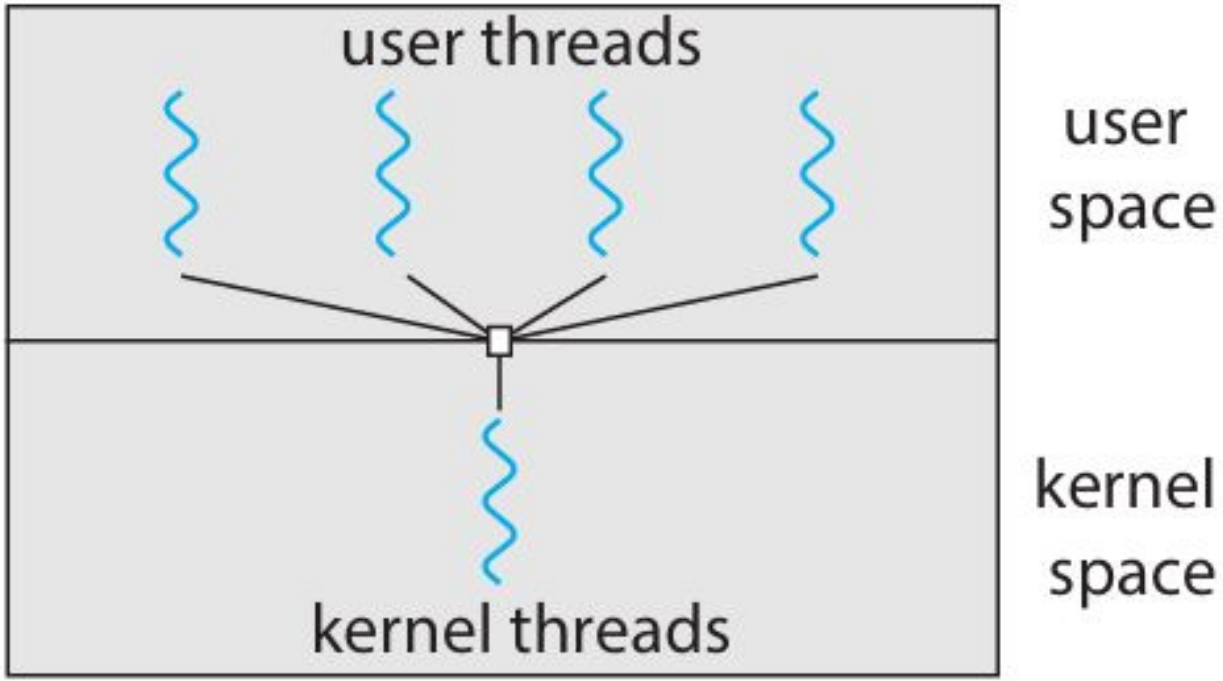


Figure 4.7 Many-to-one model.

2. One-to-One Model

- **Each user thread is mapped to a separate kernel thread**.
- Provides **better concurrency**:
  - If one thread blocks, another can run.
  - Multiple threads can run in parallel on multicore processors.
- **Disadvantage:**
  - Creating many threads increases overhead because each thread needs a kernel thread.
- Used by **Linux and Windows operating systems**.
- This is the **most commonly used model today**.

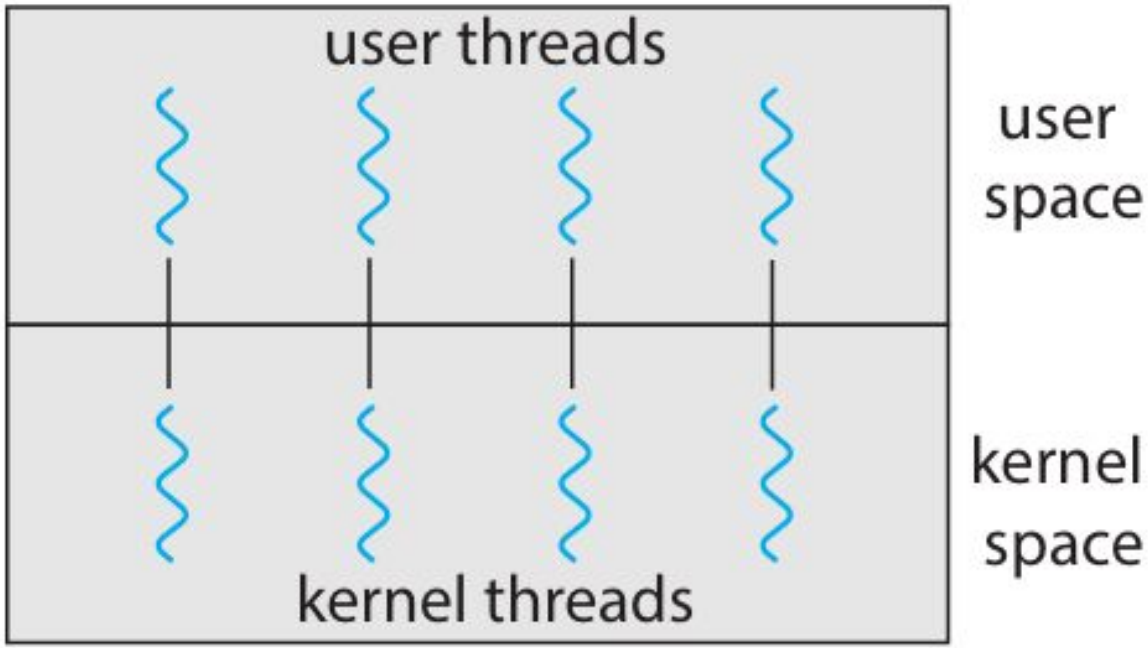


Figure 4.8 One-to-one model.

3. Many-to-Many Model

- **Many user threads are mapped to a smaller or equal number of kernel threads**.
- Combines advantages of both previous models:
  - Developers can create many user threads.
  - Kernel threads can run in parallel on multiple CPUs.
  - Blocking of one thread does not block the entire process.
- The number of kernel threads can depend on:
  - The application
  - The number of CPU cores
- **Disadvantage:** Complex to design and implement.

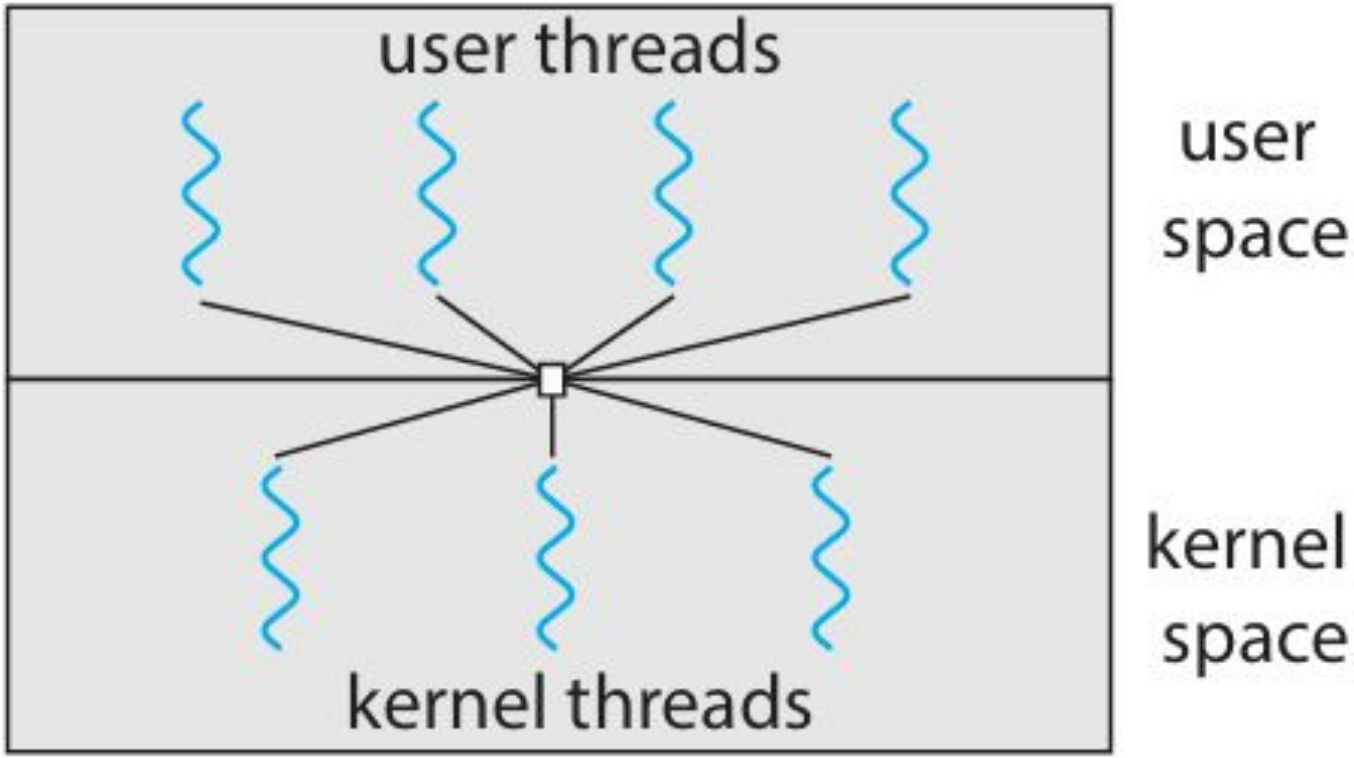


Figure 4.9 Many-to-many model.

Two-Level Model (Variation of Many-to-Many)

- Similar to the many-to-many model.
- Allows some user threads to be **bound to specific kernel threads**.
- Provides additional flexibility.

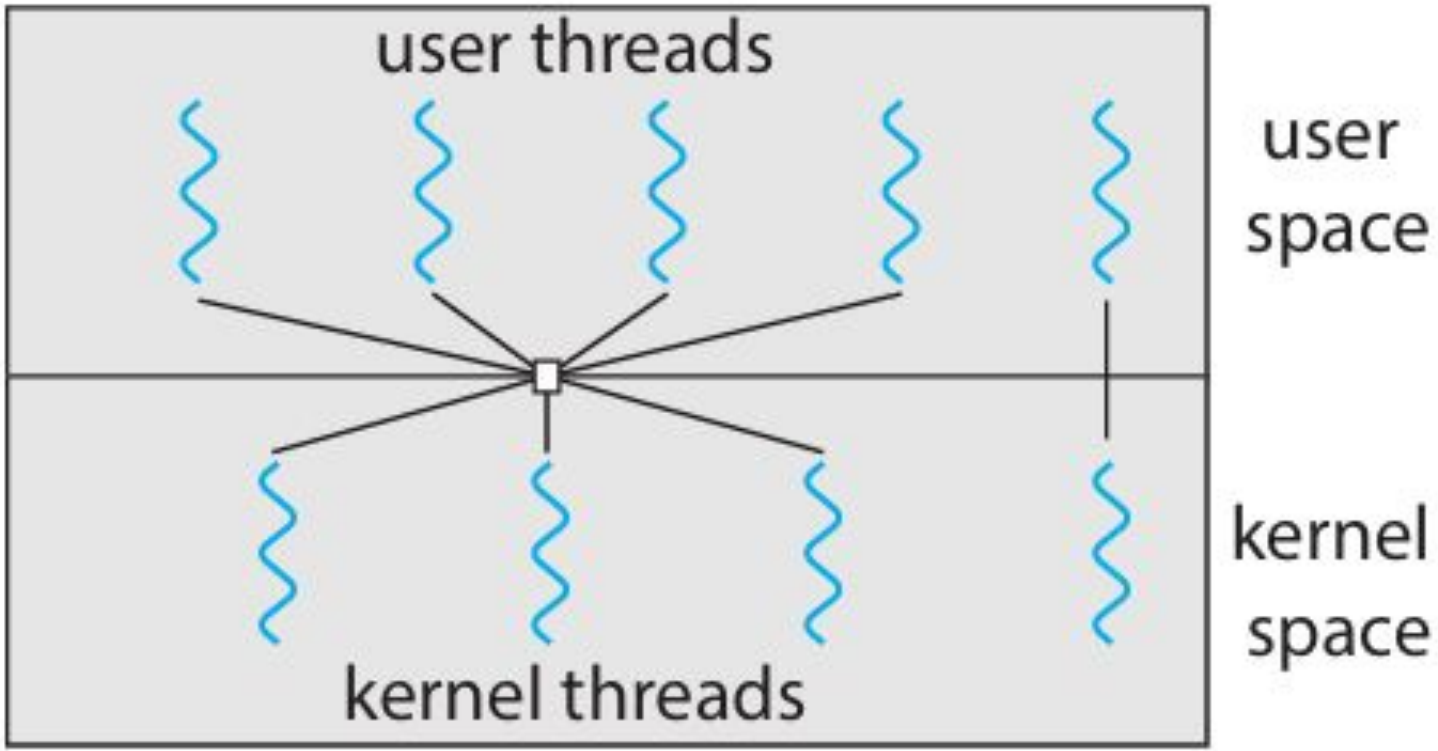


Figure 4.10 Two-level model.

Process scheduling

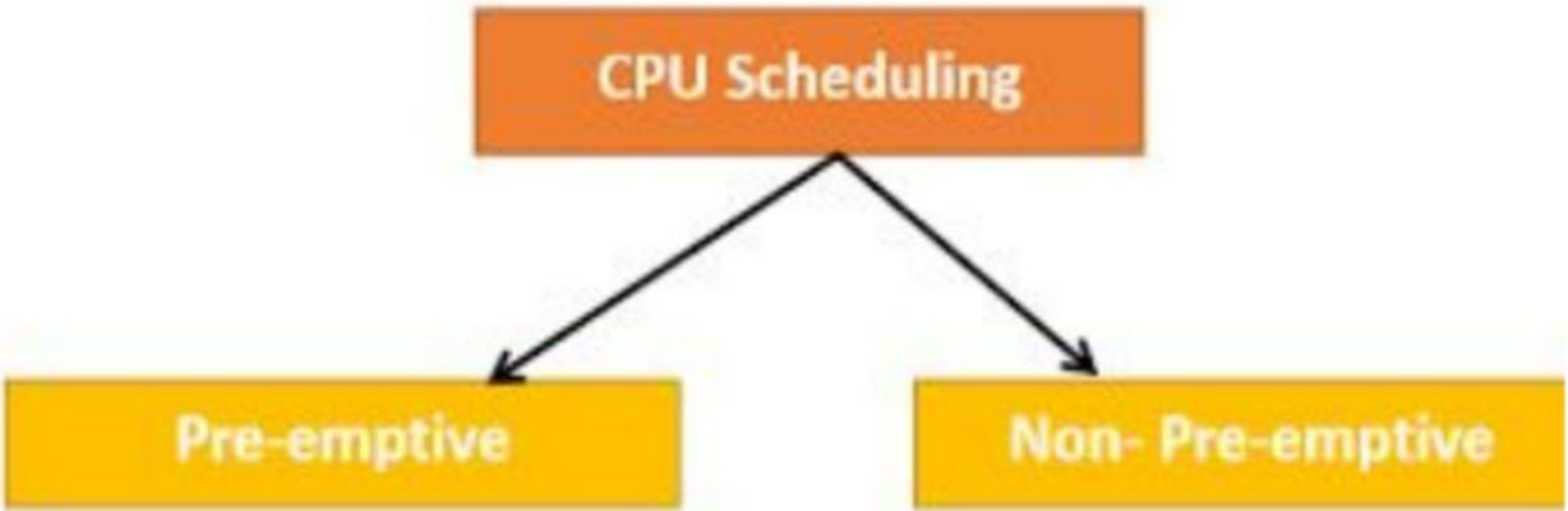
Prepared by Dilsha K P

CPU SCHEDULING

- The objective of multiprogramming is to have some process running at all times, in order to maximize CPU utilization.
- CPU scheduling is the process of deciding which process will run when multiple processes are ready

CPU Scheduler

- Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed.
- The selection process is carried out by the **short-term scheduler (or CPU scheduler).**
- The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.


CPU Scheduling
Pre-emptive
Non- Pre-emptive

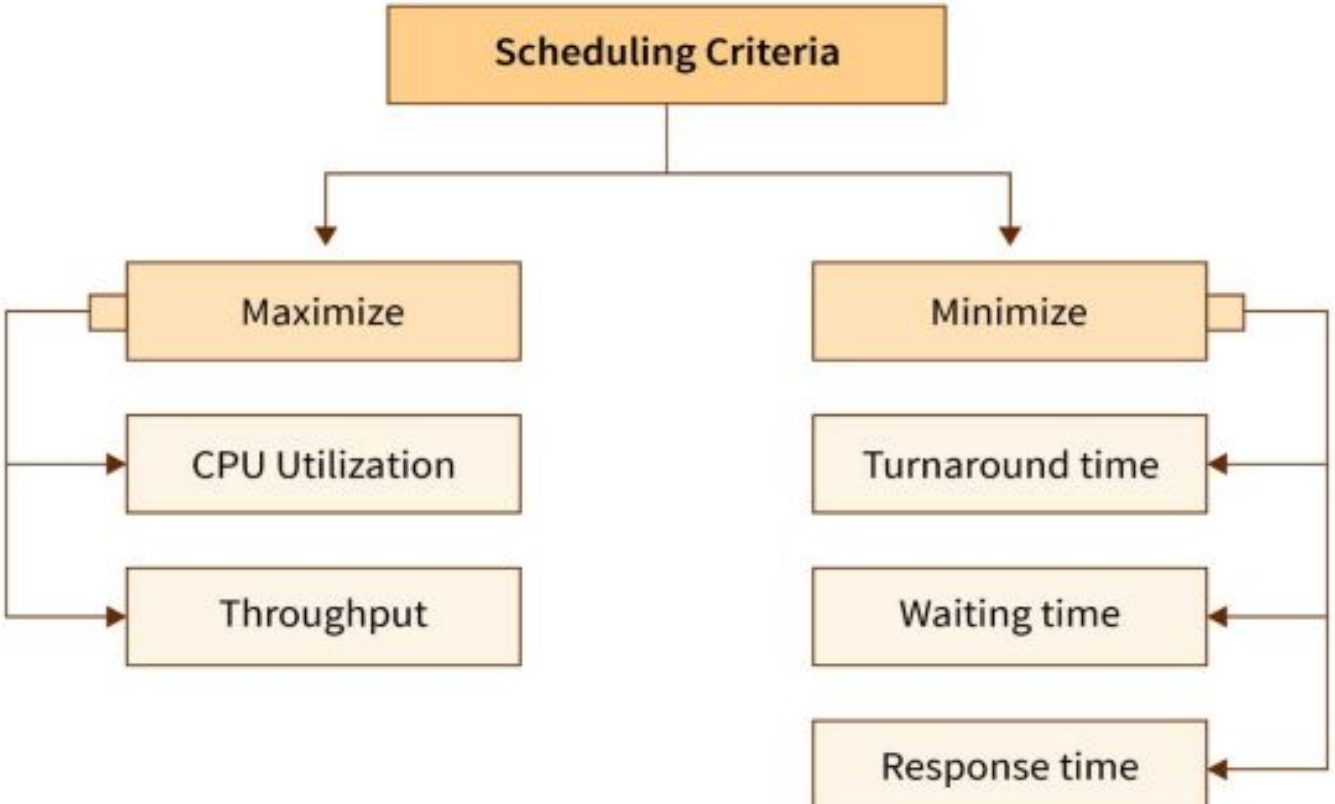
Preemptive Scheduling

- **Preemptive Scheduling** is a type of CPU scheduling in which the resources (CPU Cycle) have been allocated to a process for a limited amount of time.
- In this type of scheduling, a process can be interrupted when it is being executed.
- In preemptive scheduling, if a process that has a high priority arrives frequently in the 'ready' queue, the low priority processes may starve.

Non-Preemptive Scheduling

- **Non-Preemptive Scheduling** is one in which once the resources (CPU Cycle) have been allocated to a process, the process holds it until it completes its burst time or switches to the 'wait' state.
- In non-preemptive scheduling, a process cannot be interrupted until it terminates itself or its time is over.

SCHEDULING CRITERIA



CPU utilization:

- We want to keep the CPU as busy as possible.

CPU utilization refers to the **percentage of time the CPU is actively executing processes.**

- CPU utilization can range from **0% to 100%**:

Throughput:

- It is the number of processes completed per time unit (No. of process completed/second).

- For long processes, this rate may be 1 process per hour; for short

transactions, throughput might be 1 process per second.

Waiting Time

- measures the amount of time a task or process waits in the ready queue before it is processed by the CPU.

Turnaround time

- The interval from the time of submission of a process to the time of completion is the turnaround time.

Response time:

- it is the time from the submission of a request until the first response is produced.

Arrival Time: *Time at which the process arrives in the ready queue.*

Completion Time: *Time at which process completes its execution.*

Burst Time: *Time required by a process for CPU execution.*

Turn Around Time: *Time Difference between completion time and arrival time.*

Turn Around Time = Completion Time – Arrival Time

Waiting Time(W.T): *Time Difference between turn around time and burst time.*
Waiting Time = Turn Around Time – Burst Time

Arrival Time-AT

Burst Time-BT

Completion Time-CT

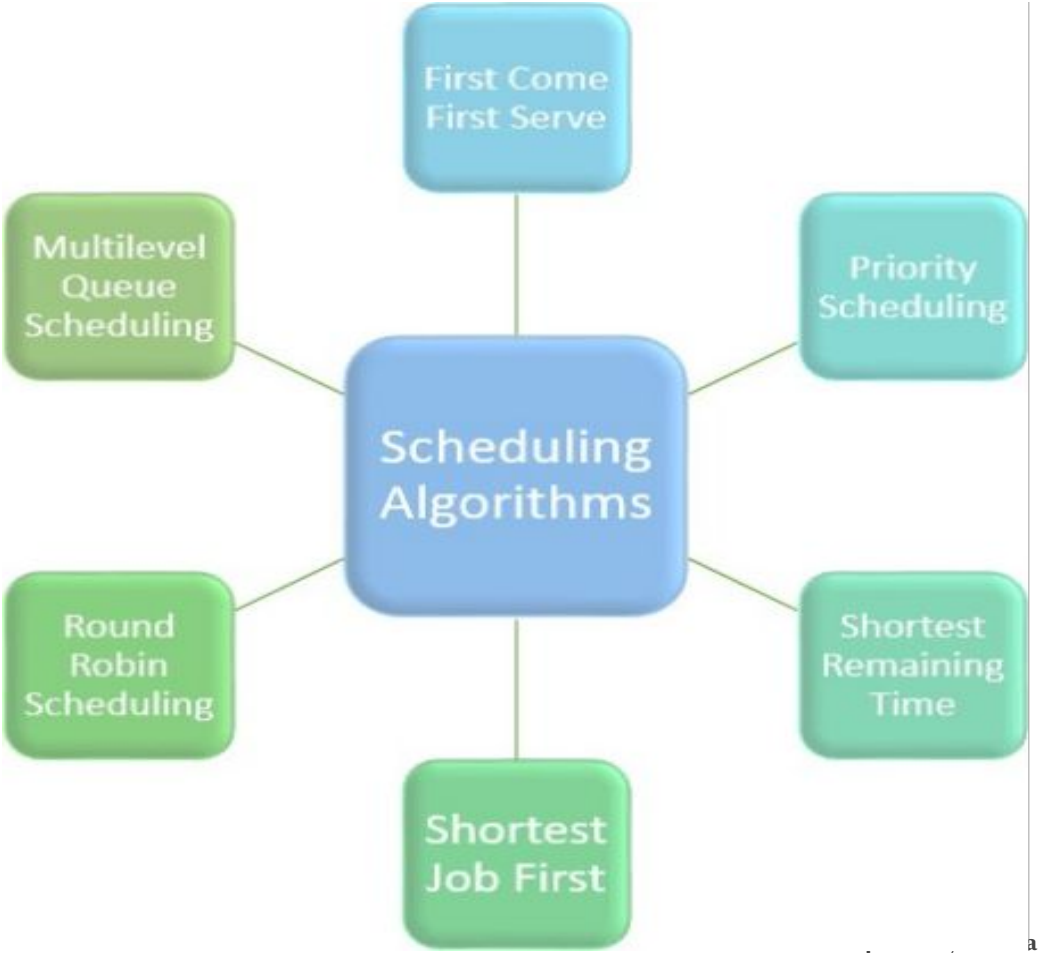
Turnaround Time-TAT(TT)

Waiting Time-WT

TT=CT-AT

WT=TT-BT

Scheduling algorithms



First come First Served (FCFS)

1. First come First Served

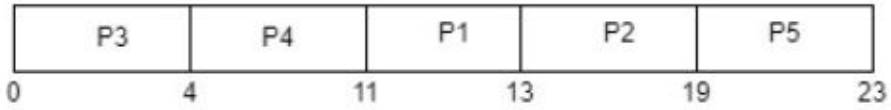
- FCFS is considered as simplest CPU-scheduling algorithm.
- In FCFS algorithm, the process that requests the CPU first is allocated in the CPU first.
- The implementation of FCFS algorithm is managed with FIFO (First in first out) queue.
- FCFS scheduling is non-preemptive.

- **Arrival time (AT)** – Arrival time is the time at which the process arrives in ready queue.
- **Burst time (BT) or CPU time of the process** – Burst time is the unit of time in which a particular process completes its execution.
- **Completion time (CT)** – Completion time is the time at which the process has been terminated.
- **Turn-around time (TAT)** – The total time from arrival time to completion time is known as turn-around time. TAT can be written as,

Consider the given table below and find Completion time (CT), Turn-around time (TAT), Waiting time (WT), Response time (RT), Average Turn-around time and Average Waiting time.

Process ID	Arrival time	Burst time
P1	2	2
P2	5	6
P3	0	4
P4	0	7
P5	7	4

Gantt chart



For this problem CT, TAT, WT, RT is shown in the given table –

Process ID	Arrival time	Burst time	CT	TAT=CT-AT	WT=TAT-BT	RT
P1	2	2	13	13-2= 11	11-2= 9	9
P2	5	6	19	19-5= 14	14-6= 8	8
P3	0	4	4	4-0= 4	4-4= 0	0
P4	0	7	11	11-0= 11	11-7= 4	4
P5	7	4	23	23-7= 16	16-4= 12	12

Average Waiting time = (9+8+0+4+12)/5 = 33/5 = 6.6 time unit (time unit can be considered as milliseconds)

Average Turn-around time = (11+14+4+11+16)/5 = 56/5 = 11.2 time unit (time unit can be considered as milliseconds)

Advantages Of FCFS Scheduling

- It is an easy algorithm to implement since it does not include any complex way.
- Every task should be executed simultaneously as it follows FIFO queue.
- FCFS does not give priority to any random important tasks first so it's a fair scheduling.

Disadvantages Of FCFS Scheduling

- FCFS results in convoy effect which means if a process with higher burst time comes first in the ready queue then the processes with lower burst time may get blocked and that processes with lower burst time may not be able to get the CPU if the higher burst time task takes time forever.
- If a process with long burst time comes in the line first then the other short burst time process have to wait for a long time, so it is not much good as time-sharing systems.
- Since it is non-preemptive, it does not release the CPU before it completes its task execution completely.

Why FIFO (FCFS) Scheduling Is Not Good

Given jobs:

- **Job A** → runs for **100 seconds**
- **Job B** → runs for **10 seconds**
- **Job C** → runs for **10 seconds**

FIFO rule:

👉 **First job arrives, runs first and completes fully** before the next job starts.

- Job **A** runs from **0–100**
- Job **B** runs from **100–110**
- Job **C** runs from **110–120**

Turnaround Time

Turnaround Time = Completion Time − Arrival Time
(Assume all arrive at time 0)

- **A** = 100 − 0 = **100**
- **B** = 110 − 0 = **110**
- **C** = 120 − 0 = **120**

Average Turnaround Time =110 seconds

➡️ **Very high average turnaround time**

Convoy Effect

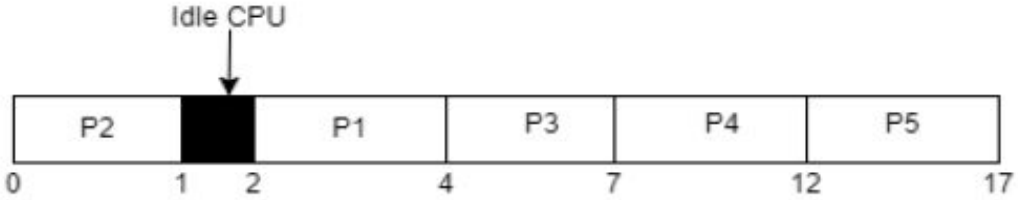
This situation is called the **Convoy Effect**.

Meaning (simple words):

- **Short jobs (B, C)** are forced to wait
- **Behind one long job (A)**
- System performance becomes poor

Consider the given table below and find Completion time (CT), Turn-around time (TAT), Waiting time (WT), Response time (RT), Average Turn-around time and Average Waiting time.

Process ID	Arrival time	Burst time
P1	2	2
P2	0	1
P3	2	3
P4	3	5
P5	4	5



For this problem CT, TAT, WT, RT is shown in the given table –

Process ID	Arrival time	Burst time	CT	TAT=CT-AT	WT=TAT-BT	RT
P1	2	2	4	4-2= 2	2-2= 0	0
P2	0	1	1	1-0= 1	1-1= 0	0
P3	2	3	7	7-2= 5	5-3= 2	2
P4	3	5	12	12-3= 9	9-5= 4	4
P5	4	5	17	17-4= 13	13-5= 8	8

Average Waiting time = (0+0+2+4+8)/5 = 14/5 = 2.8 time unit (time unit can be considered as milliseconds)

Average Turn-around time = (2+1+5+9+13)/5 = 30/5 = 6 time unit (time unit can be considered as milliseconds)

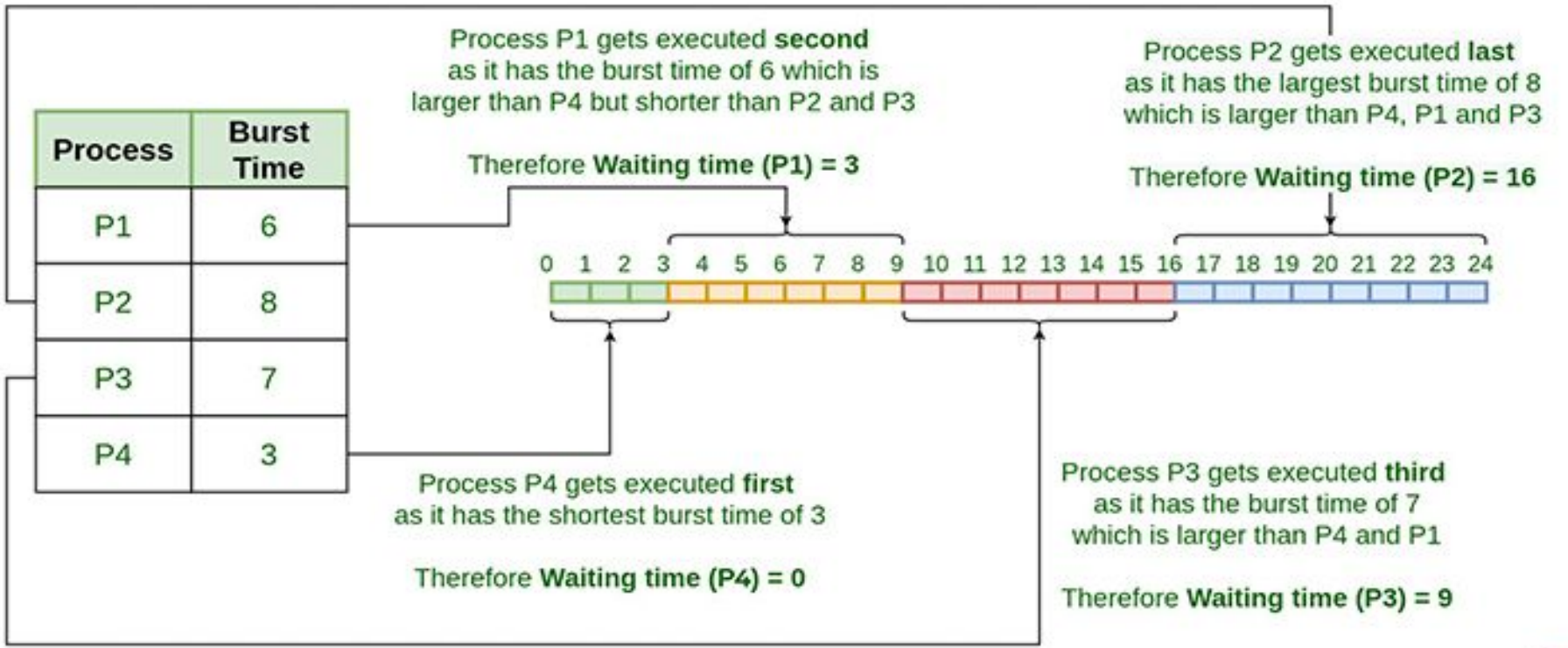
Shortest Job First

- In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.
- Both **preemptive and non-preemptive** scheduling strategies are possible in the SJF scheduling algorithm.
- **SJF is optimal:** gives minimum average waiting time for a given set of processes

Two schemes:

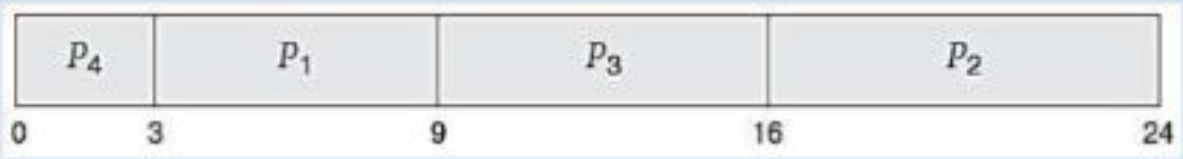
- **Non-Preemptive:** once CPU given to the process it cannot be preempted until completes its CPU burst.
- **Preemptive:** if a new process arrives with CPU burst length less than remaining time of current executing process, preempt the current process. This scheme is know as the Shortest-Remaining-Time-First (SRTF)

Shortest Job First (SJF) Scheduling Algorithm



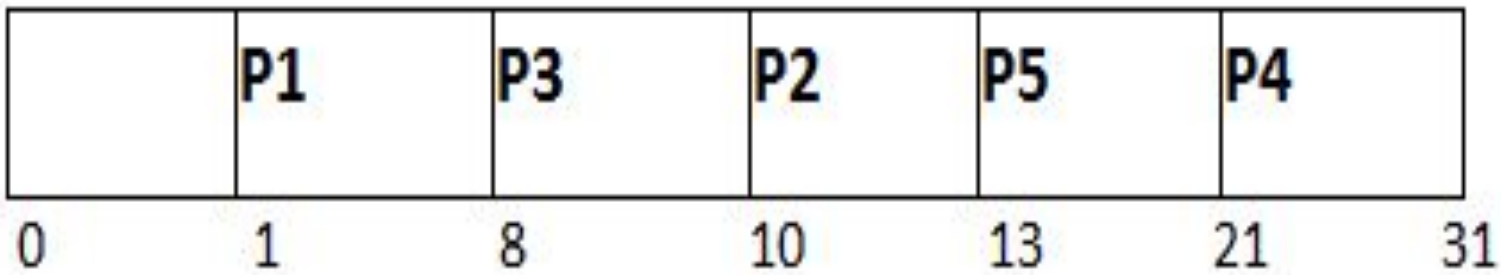
Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

• Using SJF scheduling, we would schedule these processes according to the following Gantt chart:



- The waiting time is 3 milliseconds for process P1, 16 milliseconds for process P2, 9 milliseconds for process P3, and 0 milliseconds for process P4.
- The average waiting time is **(3 + 16 + 9 + 0)/4 = 7** milliseconds.

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8


P1
P3
P2
P5
P4
0
1
8
10
13
21
31

PID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	1	7	8	7	0
2	3	3	13	10	7
3	6	2	10	4	2
4	7	10	31	24	14
5	9	8	21	12	4

Avg Waiting Time = 27/5

Shortest Remaining Time First (SRTF) Scheduling Algorithm

Prepared by Dilsha K P

Shortest Remaining Time First (SRTF) Scheduling Algorithm

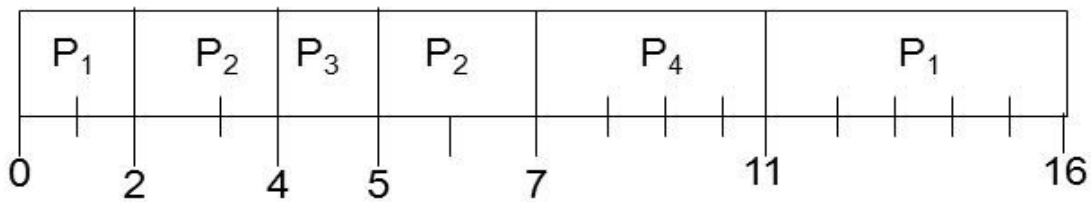
- The Preemptive version of Shortest Job First(SJF) scheduling is known as Shortest Remaining Time First (SRTF).
- In SRTF algorithm, the process having the smallest amount of time remaining until completion is selected first to execute.
- in SRTF, the processes are scheduled according to the shortest remaining time.
- SRTF algorithm involves more overheads than the SJF scheduling, because in SRTF OS is required frequently in order to monitor the CPU time of the jobs in the READY queue and to perform context switching.

Example of SRTF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

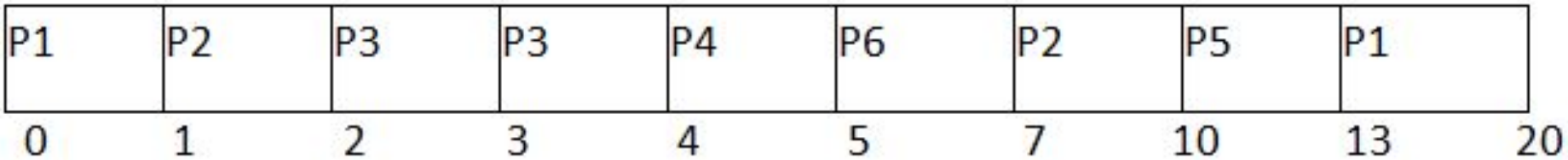
Example of SRTF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4



- **Throughput**: 4 processes/16 milliseconds = 1/4
- **Turnaround time** for P_1 = 16-0 = 16; P_2 = 7-2 = 5; P_3 = 5-4 = 1; P_4 = 11-5 = 6
- **Average turnaround time**: (16 + 5 + 1 + 6)/4 = 28/4 = 7
- **Waiting time** for P_1 = 11-2 = 9; P_2 = 5-4 = 1; P_3 = 4-4 = 0; P_4 = 7-5 = 2
- **Average waiting time**: (9 + 1 + 0 +2)/4 = 12/4 = 3

Process ID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	2
4	3	1
5	4	3
6	5	2

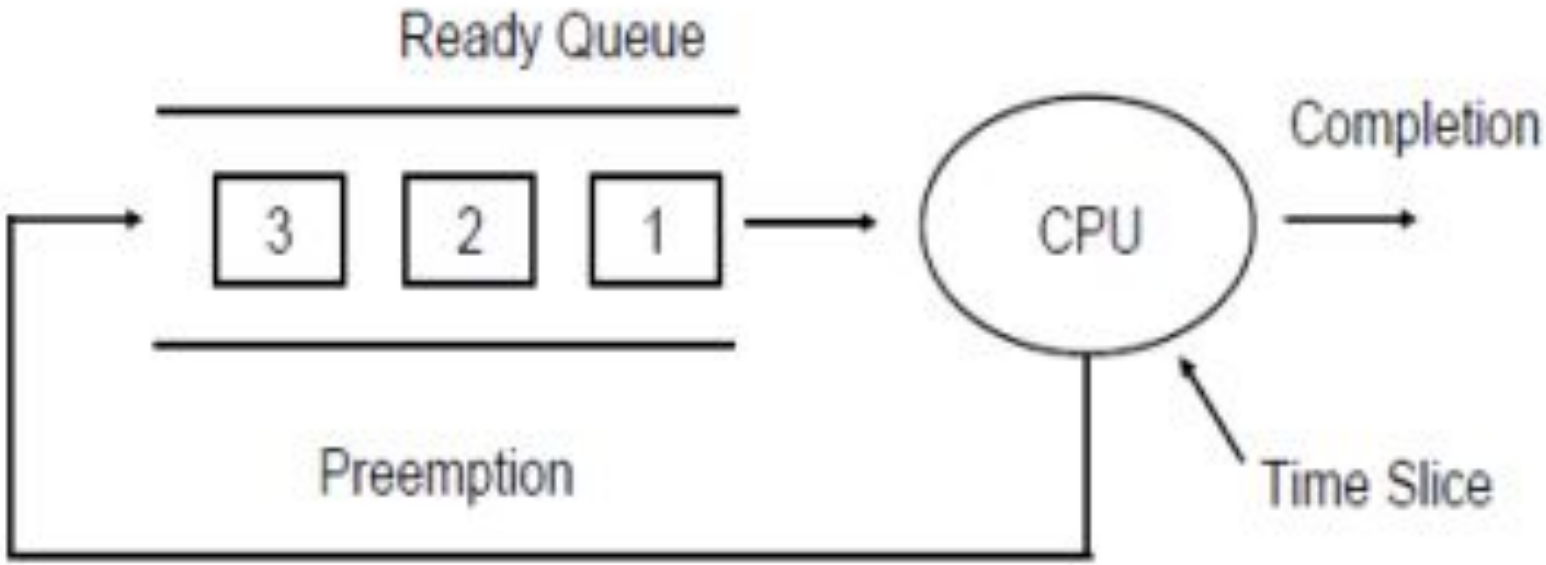

P1
P2
P3
P3
P4
P6
P2
P5
P1
0
1
2
3
4
5
7
10
13
20

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	8	20	20	12
2	1	4	10	9	5
3	2	2	4	2	0
4	3	1	5	2	1
5	4	3	13	9	6
6	5	2	7	2	0

Avg Waiting Time = 24/6

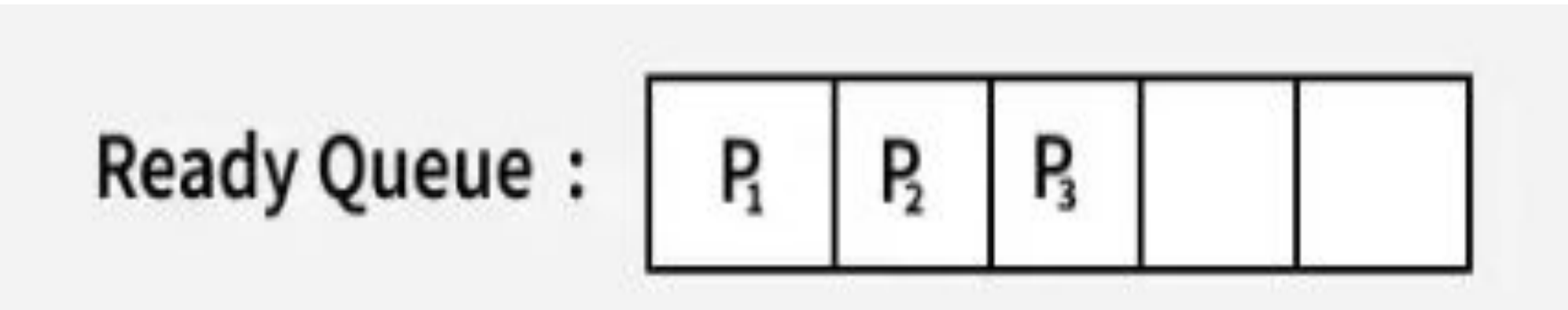
RoundRobin

- The Round robin scheduling algorithm is one of the CPU scheduling algorithms in which every process gets a fixed amount of time quantum to execute the process.
- A certain time slice is defined in the system which is called time **quantum**.
- Each process present in the ready queue is assigned the CPU for that time quantum, if the execution of the process is completed during that time then the process will **terminate** else the process will go back to the **ready queue** and wait for the next turn to complete the execution.
- It is a Preemptive type of CPU scheduling algorithm in OS.


Ready Queue
3
2
1
CPU
Completion
Preemption
Time Slice

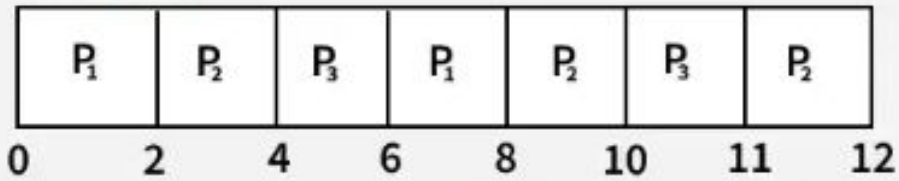
Time quantum = 2ms

Process	Arrival Time	Burst Time
P_1	0 ms	4 ms
P_2	0 ms	5 ms
P_3	0 ms	3 ms


Ready Queue :
P1
P2
P3

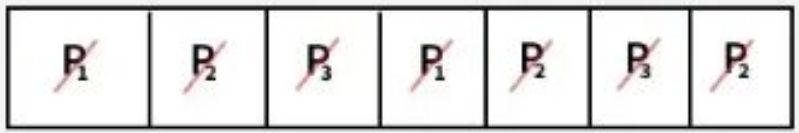
at t = 12

Gantt chart



Remaining time left(P_2) : 0 ms

Ready Queue :



- ***Turnaround Time*** *= Completion Time - Arrival Time*
- ***Waiting Time*** *= Turnaround Time - Burst Time*

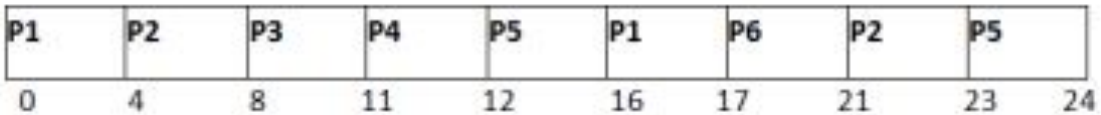
Processes	AT	BT	CT	TAT	WT
P1	0	4	8	8-0 = 8	8-4 = 4
P2	0	5	12	12-0 = 12	12-5 = 7
P3	0	3	11	11-0 = 11	11-3 = 8

- **Average Turn around time** = (8 + 12 + 11)/3 = 31/3 = 10.33 ms
- **Average waiting time** = (4 + 7 + 8)/3 = 19/3 = 6.33 ms

In the following example, there are six processes named as P1, P2, P3, P4, P5 and P6. Their arrival time and burst time are given below in the table. The time quantum of the system is 4 units

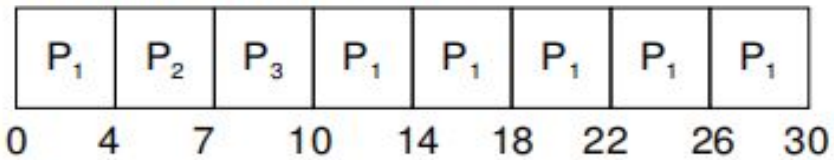
Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	5	17	17	12
2	1	6	23	22	16
3	2	3	11	9	6
4	3	1	12	9	8
5	4	5	24	20	15
6	6	4	21	15	11

Avg Waiting Time = (12+16+6+8+15+11)/6 = 76/6 units



Process	Burst Time
P1	24
P2	3
P3	3

- **Quantum time = 4 milliseconds**
- **The Gantt chart is:**



- **Average waiting time = {[0+(10-4)] +4+7}/3 = 5.6**

Q:Find the average turn around time and waiting time

Process No.	Arrival Time	Burst Time
P1	0	5
P2	1	4
P3	2	2
P4	4	1

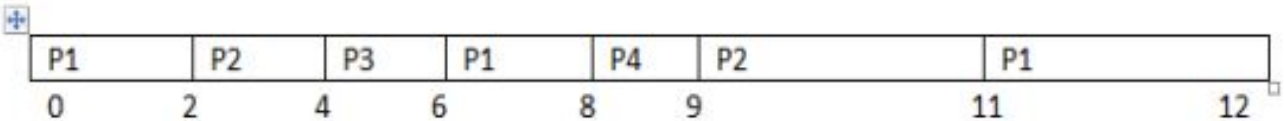
Time Quantum-2

Process No.	Arrival Time	Burst Time	Completion Time	TAT(CT-AT)	Waiting Time(TAT-BT)	Response Time
P1	0	5	12	12	7	0
P2	1	4	11	10	6	1
P3	2	2	6	4	2	2
P4	4	1	9	5	4	4

Ready Queue:

P1	P2	P3	P1	P4	P2	P1	

GANTT CHART:



Advantages

1. It can be actually implementable in the system because it is not dependent on the burst time.
2. It doesn't suffer from the problem of starvation or convoy effect.
3. All the jobs get a fair allocation of CPU.

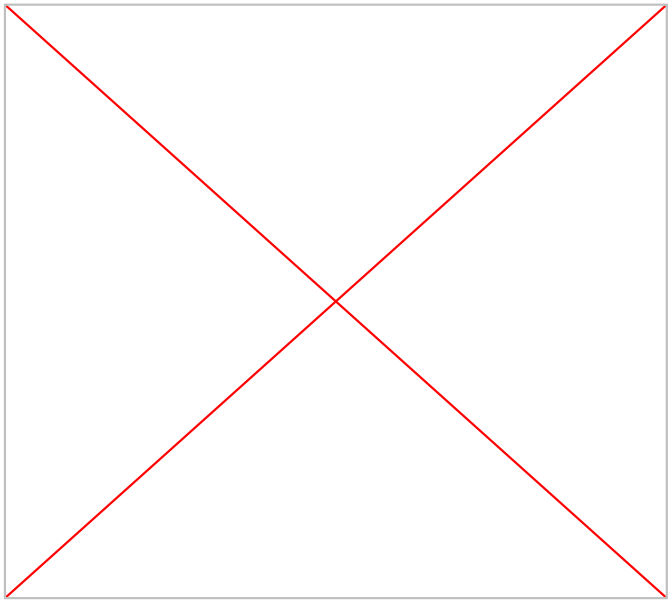
Disadvantages

1. The higher the time quantum, the higher the response time in the system.
2. The lower the time quantum, the higher the context switching overhead in the system.
3. Deciding a perfect time quantum is really a very difficult task in the system.

Multilevel Feedback Queue (MLFQ)

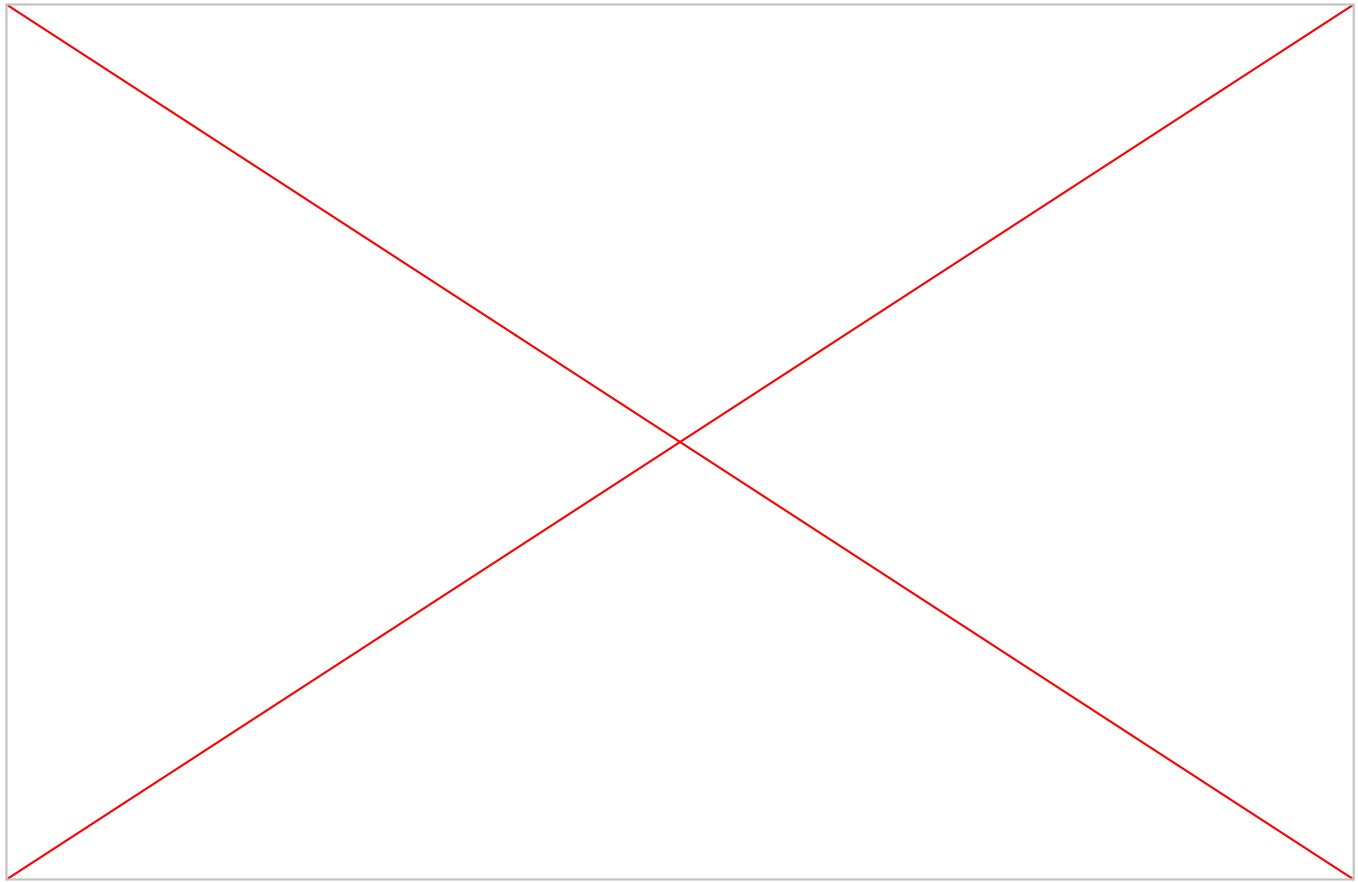
Multilevel Feedback Queue (MLFQ) Scheduling

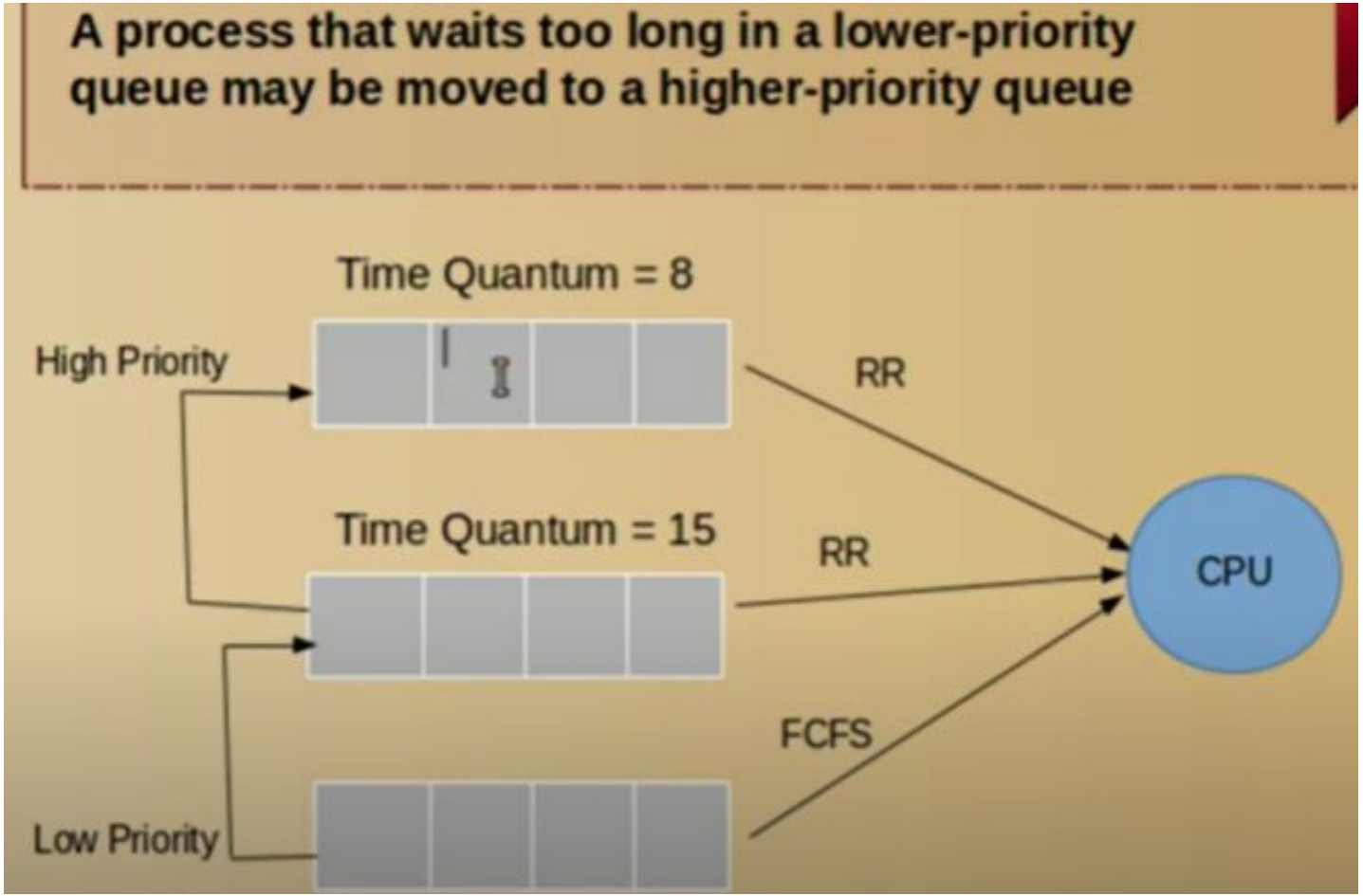
- **Multilevel Feedback Queue Scheduling (MLFQ)** CPU Scheduling is like Multilevel Queue(MLQ) Scheduling but in this process can move between the queues. And thus, much more efficient than multilevel queue scheduling.



Key Parameters:

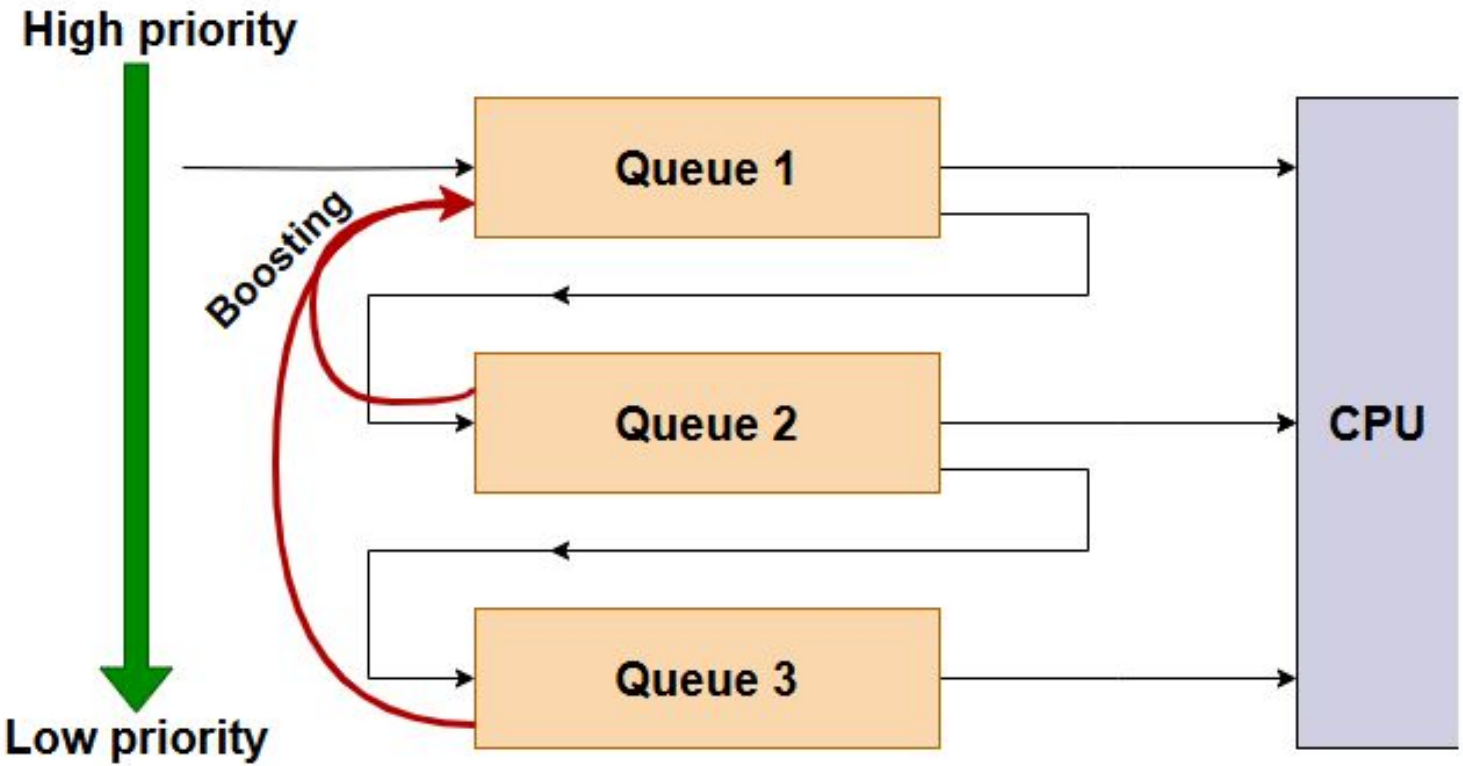
1. **Number of Queues:** Defines how many priority levels exist.
2. **Scheduling Algorithm for Each Queue:** Different queues may use different scheduling methods (e.g., Round Robin for interactive tasks, FCFS for batch jobs).
3. **Promotion Criteria:** Determines when a process should move to a higher-priority queue.
4. **Demotion Criteria:** Determines when a process should move to a lower-priority queue.
5. **Queue Assignment Rules:** Defines which queue a process enters when it arrives.




A process that waits too long in a lower-priority queue may be moved to a higher-priority queue
Time Quantum = 8
High Priority
RR
Time Quantum = 15
RR
CPU
FCFS
Low Priority

The steps involved in MLFQ scheduling when a process enters the system.

1. When a process enters the system, it is initially assigned to the highest priority queue.
2. The process can execute for a specific time quantum in its current queue.
3. If the process completes within the time quantum, it is removed from the system.
4. If the process does not complete within the time quantum, it is demoted to a lower priority queue and given a shorter time quantum.
5. This promotion and demotion process continues based on the behavior of the processes.
6. The high-priority queues take precedence over low-priority queues, allowing the latter processes to run only when high-priority queues are empty.
7. The feedback mechanism allows processes to move between queues based on their execution behavior.
8. The process continues until all processes are executed or terminated.


High priority
Queue 1
Boosting
Queue 2
CPU
Queue 3
Low priority